

The Agentic Architect Playbook

Managing the Enterprise Transformation in the
Intelligence Revolution age



Carmelo Iaria

Founder, Synaptic AI Consulting



Poll Question - Ice Breaker

What is your experience level with this topic?
(Single response)

1. Well-versed in the topic area
2. Some familiarity with the topic area
3. New to the topic area



Your Learning Journey

Course Overview

01

Module 1: Foundations

The role, the market, the skills

02

Module 2: Governance & Control

How to build guardrails for agent systems

03

Module 3: Observability & Quality

How to measure and optimize

04

Module 4: Delivery & Rollout

The blueprint and the capstone

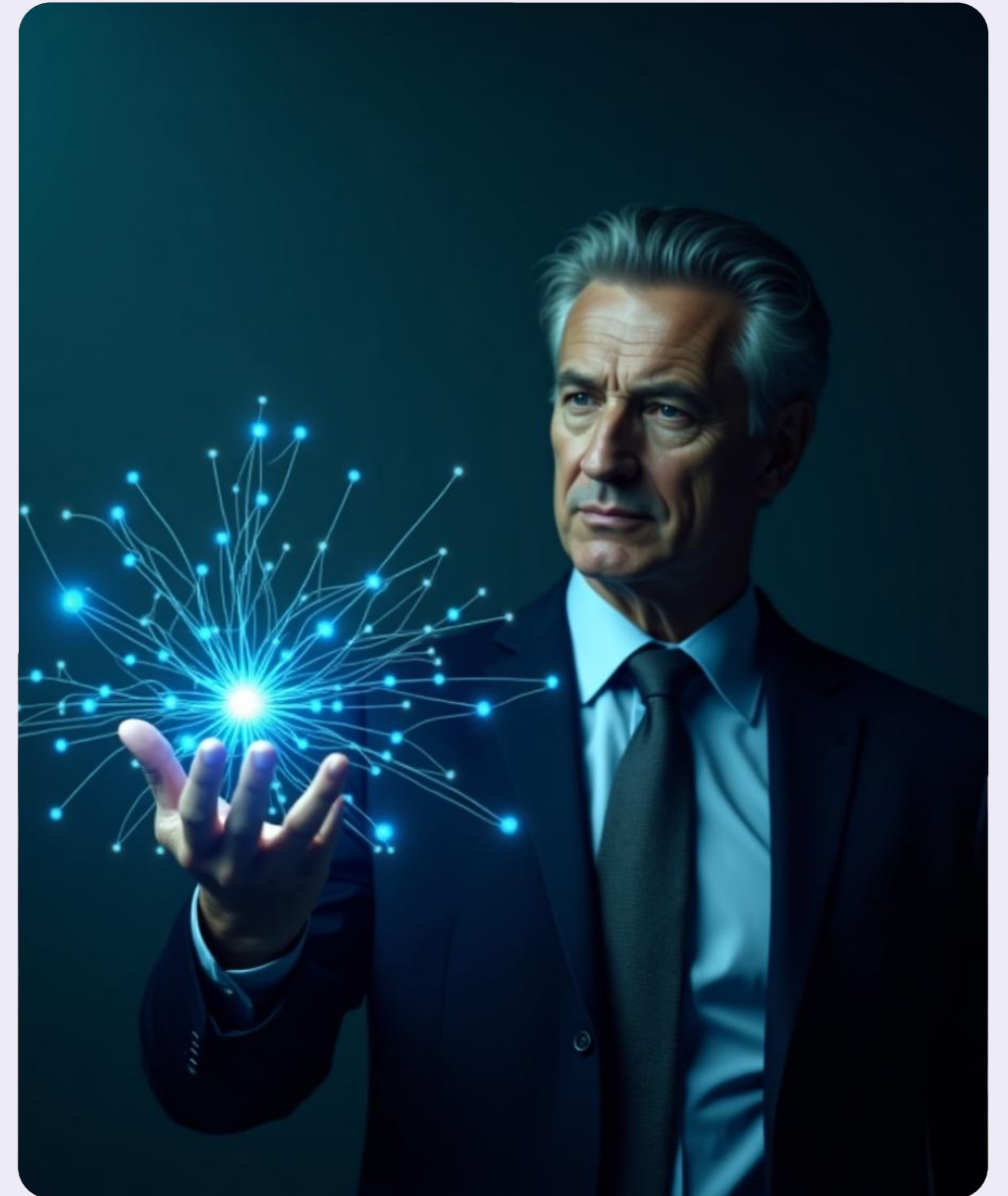




Carmelo Iaria - The Agentic Architect Playbook

Module 1

Agentic Architect Foundations



From Steam to Silicon: The Next Great Transformation



The Industrial Revolution (1760s-1840s)

This era was defined by groundbreaking technological advances that mechanized production and reshaped society.

- Mass production & urbanization
- New global trade routes
- Transformation of labor markets



The Intelligence Revolution (2020s onwards)

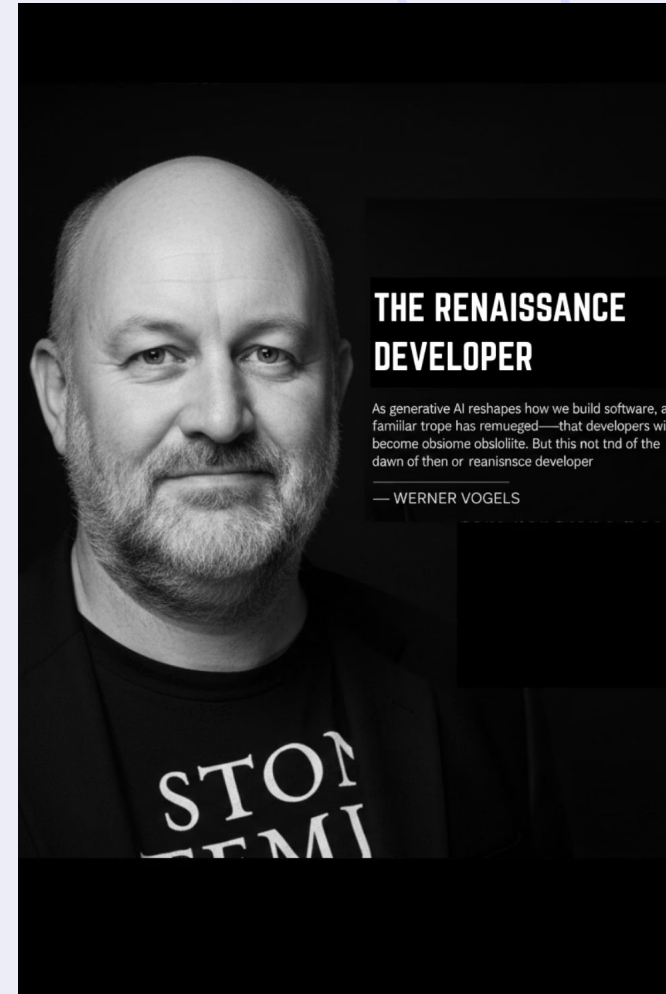
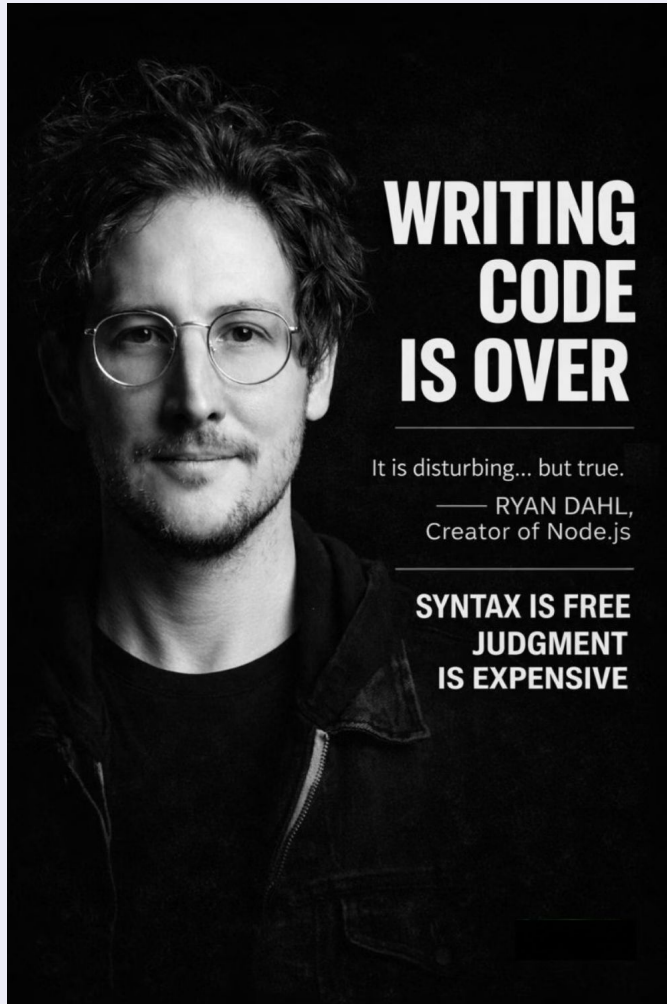
We are now entering a similar period, driven by AI, cognitive automation, and agentic systems.

- Cognitive automation & personalized production
- Restructuring of global value chains
- Transformation of knowledge work



Building Software is Fundamentally Changing

but not disappearing ...



The Productivity Paradox

Why AI Productivity Gains Haven't Translated to Business Value

Key Statistics

90%

Software professionals
now use AI tools

30-60%

Productivity gains
reported in automated workflows

35-55%

Development velocity
increases documented

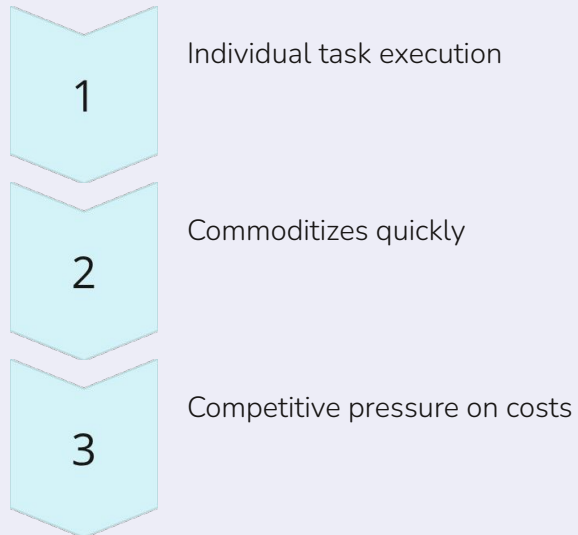
- ❏ **The Gap:** Individual task optimization ≠ System-level performance
- Organizations struggle to translate speed into proportional business value



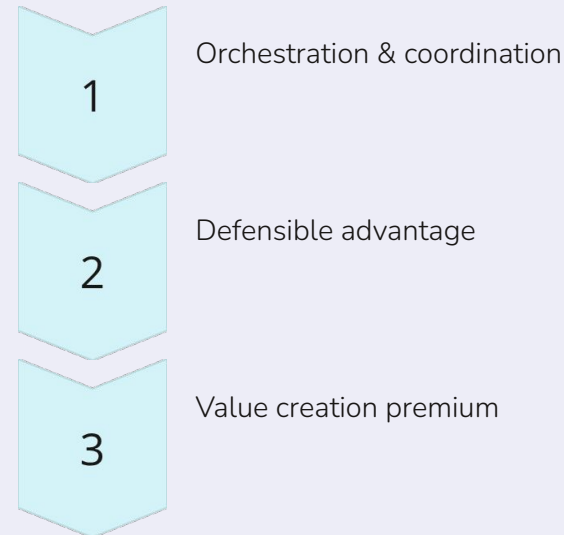
Where is the Economic Value Moving to?

From Task Execution to Orchestration & Coordination

OLD MODEL



NEW MODEL



Operate “Above the Algorithm”

Don't fall into the Vibe Coding trap

Above the Algorithm

(Agentic Architect)

- Design coordination frameworks
- Manage complexity and risk
- Apply judgment, strategic thinking, systems vision
- Create irreplaceable value

Below the Algorithm

(Vibe Coding)

- Use AI for faster task execution
- Competing in a race you can't win
- Ultimately replaceable



Who is the Agentic Architect ?

A new Role for a new Era

Definition

The Agentic Architect orchestrates AI agents and human teams to build products in an AI-native manner. This role generates value through **coordination, design, and governance**—not necessarily through individual task execution.



Three Key Capabilities:

Bridge Vision & Reality

Translate business objectives into multi-agent system architectures

Design Systems

Coordinate specialized AI agents, human reviewers, and external systems

Govern Risk

Manage access control, compliance, and ethical AI behavior



What Makes it Different ?

Agentic Architect ≠ Other Technical Roles

Role	Focus	Value Creation
Software Engineer	Implementation, code	Task execution
Product Manager	Requirements, strategy	Feature definition
AI/ML Engineer	Models, training	Algorithm development
Enterprise Architect	Infrastructure, systems	Platform design
Agentic Architect	Orchestration, governance, systems	Coordination at scale

The Key Distinction:

- Engineers focus on "how to build X"
- Agentic Architects focus on "how to coordinate agents building X"
- Not replacing these roles—enabling them at scale



The “Three Hats” Model

Agentic Architects Wear Three Integrated Hats

Technology

- Working mental model of what agents can and cannot do
- Familiarity with orchestration patterns (sequential, hierarchical, concurrent)
- Understanding of context management and memory architectures
- Knowledge of evaluation frameworks and testing strategies

Experience & Design

- Voice, tone, personality of agents (formal vs. approachable)
- How agents handle ambiguity, errors, and escalations
- User interaction patterns and conversation design
- Balancing autonomy with human oversight

Business Outcomes

- Cost reduction (automation replacing manual work)
- Revenue generation (improved conversion, personalization)
- Customer satisfaction (faster resolution, 24/7 availability)
- Risk mitigation (compliance, quality assurance)
- Competitive advantage (time-to-market)



Integration: All three hats working together

- Technology without design = powerful but unusable systems
- Design without business outcomes = beautiful but valueless systems
- Business outcomes without governance = risky, non-compliant systems



The Seven Core Competencies

Mastering the Art of Agentic Architecture

1

Systems Thinking

Design holistic solutions by understanding how AI agents, humans, and existing systems interact as a cohesive whole.

2

Context Engineering

Craft precise instructions and structure information effectively for optimal AI agent input, processing, and output.

3

Orchestration Patterns

Apply various architectural patterns (sequential, hierarchical, concurrent) to coordinate complex multi-agent workflows.

4

Human-in-the-Loop Design

Strategically integrate human oversight and decision points to ensure ethical behavior and high-quality outcomes.

5

Evaluation & Quality Assurance

Develop robust metrics and processes to continuously assess AI agent performance, reliability, and output quality.

6

Governance & Risk Management

Establish policies, controls, and ethical frameworks to manage access, compliance, and mitigate potential risks.

7

Business Acumen

Translate business objectives into AI strategies, articulate value, and align technical solutions with organizational goals.



Core Competency #1

Systems Thinking

Seeing Beyond Components to
Emergent Behavior

Definition

Understanding how multiple agents interact, create emergent behaviors, and collectively deliver business outcomes

Key Skills



Identifying bottlenecks

where coordination (not speed) limits performance



Mapping dependencies

between agents, data sources, APIs, human decision points



Anticipating failure modes

and designing graceful degradation patterns



Balancing trade-offs

between autonomy/control, speed/accuracy, cost/capability

✗ Bad Systems Thinking:

Optimize first-response time for customer support agents

✓ Good Systems Thinking:

Recognize that fast first-response without proper escalation handoffs and knowledge base quality creates poor system-level outcomes



Core Competency #2

Context Engineering

Managing Information,
not just prompting

Definition

The discipline of managing what information AI agents receive to enable better decisions

Key Techniques



Writing Context

Save information outside context window (memory objects, databases) so agents reference it later without token overhead



Compressing Context

Summarize interactions, research findings into dense, information-rich artifacts



Selecting Context

Use RAG, semantic search, filtering to surface only relevant information at decision time



Isolating Context

Give each agent scoped context; sub-agents handle specific tasks returning only essential findings

The "Dumb Zone" Principle:

Beyond ~40% context utilization, agent performance degrades significantly. Keep agents in the "smart zone" through intentional compaction and sub-agent delegation.



Core Competency #3

Orchestration Patterns

Well Orchestrated Multiagent
Systems Deliver Speed

Definition

Knowing When to Use Which Agents Coordination Strategy

Main Patterns



Sequential

Agents execute in order, each building on results

Best for: Multistage workflows, progressive refinement

Example: Research → Plan → Implement → Test → Deploy



Concurrent

Agents work in parallel, reducing latency

Best for: Independent subtasks, parallelizable analysis

Example: Market research (competitor + sentiment + pricing in parallel)



Handoff

Agents route to more appropriate specialists dynamically

Best for: Triage scenarios (optimal agent unknown upfront)

Example: Customer query → routing to billing/sales agent



Hierarchical

Manager agent plans, delegates to specialized workers

Best for: Complex problems, iterative planning

Example: Incident response coordinator managing diagnostic, infrastructure, communication agents

Pattern Combination

Real systems blend patterns—sequential core with concurrent analysis and hierarchical coordination



Core Competency #4

Human-In-The-Loop Design

Keeping Humans Appropriately
Engaged with Agentic Systems

Design Principles:

- Focus on **decision points**, not prompts—where does human judgment matter most?
- **Categorize risk actions**—automate low-risk, escalate high-impact
- Provide **lightweight context** to reviewers, not raw data dumps
- Use **policy-driven** enforcement, not hardcoded if-statements
- Enable **async approvals** when real-time response unnecessary

Five Key HITL Mechanisms:



Interrupt Gates

Pause before high-stakes actions (transactions, deletions, configs) requiring explicit approval



Approval Workflow

Route specific actions through policy-based approval layers with role-based permissions



Human-as-Tool

Agents invoke "human" tool when uncertain, receiving guidance to continue



Fallback Escalation

Agent failures trigger escalation to humans via Slack, email, dashboards



Audit-First Architecture

Every decision logged for review, compliance, continuous improvement

Why is Critical

- Governance without HITL = false confidence
- HITL without automation = no value



Core Competency #5

Evaluation & Quality Assurance

Testing Non-Deterministic Systems at Scale

5 Evaluation Layers:

- **Model Layer:** Accuracy, factual correctness, hallucination rates, reasoning transparency
- **Agent Layer:** Plan quality, tool selection correctness, argument generation accuracy
- **System Layer:** Task completion rates, multi-step success tracking, latency, resource utilization
- **Business Layer:** User satisfaction (NPS/CSAT), resolution quality, cost per interaction, revenue impact
- **Safety & Trust:** Bias detection, harmful content prevention, policy compliance, explainability

Evaluation Strategies:



Automated Testing

Statistical evaluators, programmatic checks, regression testing



LLM as a Judge

AI evaluating AI outputs to compare approaches and flag regressions



Simulation Harness

Synthetic personas running thousands of pre-production scenarios



Continuous Monitoring

Real-time tracking of production performance and business KPIs



Human Review

Validation of high-stakes outputs and complex edge cases



Why is Critical

Ensure the AI system remains accurate, reliable, safe, and aligned with business goals across every layer — from model performance to user trust.



Core Competency #6

Governance & Risk Management

Building Compliance Into
Agentic Systems

Design Principle

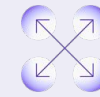
Embed compliance, accountability, and ethical safeguards directly into system architecture—ensuring safety, trust, and resilience by design, not as an afterthought.

Four Pillars



Access Control

- Role-based permissions for agent tool access
- Least-privilege principles
- Segregation of duties



Policy Enforcement

- Compliance with regulations (GDPR, HIPAA, SOC 2)
- Business rule validation
- Brand guideline adherence
- Ethical AI principles



Risk Mitigation

- Deterministic workflows for critical paths
- Confidence thresholds for escalated decisions
- Rollback capabilities and version control
- Incident response procedures



Observability

- Comprehensive logging and audit trails
- Performance dashboards and alerting
- Anomaly detection
- Root cause analysis tooling

Why Governance First Matters:

- Governance bolted on afterward = always inadequate
- Governance built in = becomes competitive advantage
- Compliance becomes enabler, not bottleneck



Core Competency #7

Business Acumen

Translating Technical Capabilities
into Business Value

Three Dimensions



Value Articulation

- Quantifying ROI (productivity gains, cost reduction, revenue increase, risk mitigation)
- Identifying high-impact use cases (6-10x returns)
- Aligning with organizational strategy
- Managing stakeholder expectations



Financial Modeling

- Understanding payback periods (typically 6-12 months for agentic AI)
- Calculating TCO (infrastructure, development, operations, governance)
- Forecasting scaling economics
- Benchmarking against industry standards (30-60% productivity gains, 171% average ROI)



Change Management

- Redesigning workflows around agentic capabilities
- Retraining teams for new roles (agent coaches, reviewers, orchestrators)
- Building trust through transparency and demonstrated reliability
- Creating champions who advocate for agentic approaches



Why This Matter Now

The Window of Opportunity Is Open, But Closing

Current Market Dynamics (2026):

- AI capabilities approaching human parity on many tasks
- Talent market shifting from "AI knowledge" to "AI orchestration skills"
- First-mover advantage in agentic patterns lasting 6-18 months
- Organizations adopting structured approaches (AAMAD, frameworks) outpacing ad-hoc experimentation 3:1

Gartner Prediction

By 2028:

One-third of enterprise applications will embed agentic AI

15% of routine decisions made autonomously

Key Benefits:

Quality improves while speed increases
(not a trade-off)

Junior developers become productive faster
(following established patterns)

Senior developers focus on architecture vs.
implementation details



Skills Development & Career Progression

From Student to Thought Leader

Level 1: Practitioner

(Months 1-3)

- Execute structured projects using AAMAD methodology
- Implement orchestration patterns
- Apply context engineering basics
- Use evaluation frameworks

Target role:

Specialist/IC contributing to agentic initiatives

Level 2: Architect

(Months 4-9)

- Design custom multi-agent architectures for complex problems
- Create organization-specific patterns and templates
- Optimize context strategies for domain expertise
- Lead pilot projects and mentor junior practitioners

Target role:

Senior IC or team lead

Level 3: Strategist

(Months 10-18)

- Align agentic initiatives with organizational strategy
- Design enterprise-wide governance frameworks
- Drive organizational transformation and change management
- Contribute to framework evolution and industry best practices

Target role:

Director or principal architect

Level 4: Thought Leader

(18+ months)

- Publish patterns and insights to broader community
- Speak at conferences and write authoritative content
- Consult on large-scale agentic transformations
- Innovate novel orchestration paradigms

Target role:

VP/Executive, independent advisor, or founder



The Agentic Architect Playbook

How maintain your Economic Value in the
age of AI Assisted Code Development



Customer Support Multiagent System

Our Capstone Use Case

CAPSTONE USE CASE

The Scenario

A mid-market bank wants to automate 70% of customer support through a multi-agent system while maintaining human oversight for complex cases.

What We'll Build:

01	02	03	04
Module 1	Module 2	Module 3	Module 4
Architecture blueprint showing the three hats in action	Governance control map - how do we prevent agent mistakes?	Observability dashboard - how do we measure quality and business impact?	One-page rollout blueprint and governance checklist - complete capstone

Your Job

- Follow along to understand the framework
- Map your own organizational challenge to this structure
- By Module 4, you'll have a template for your context



Customer Support Multiagent System

Our Blueprint (the three hats in action)

CAPSTONE USE CASE

Hat 1: Technology

Agents needed: Triage agent, resolution agent, escalation agent, knowledge manager

Orchestration pattern: Hierarchical with handoff (triage routes to appropriate specialist)

Context management: Knowledge base integration, customer history, company policies

Evaluation: First-contact resolution rate, escalation accuracy, response time

Hat 2: Experience & Design

Voice/tone: Empathetic, professional, appropriate to bank context

Handling escalations: When to route to human vs. attempt resolution

User patterns: Mobile-first interaction, multiple channels (chat, email)

Autonomy balance: Resolve simple inquiries automatically, escalate complex cases

Hat 3: Business Outcomes

Cost reduction: \$180K+ annually (70% automation target, 24/7 operation)

Customer satisfaction: NPS improvement through faster resolution

Risk mitigation: Compliance with banking regulations, audit trails

Competitive advantage: Time-to-market for new support features

Integration (Why All Three Matter):

- Technology alone = capable but possibly noncompliant agents
- Experience alone = beautiful but ineffective interactions
- Business outcomes alone = aggressive targets with risky shortcuts
- Together = sustainable competitive advantage

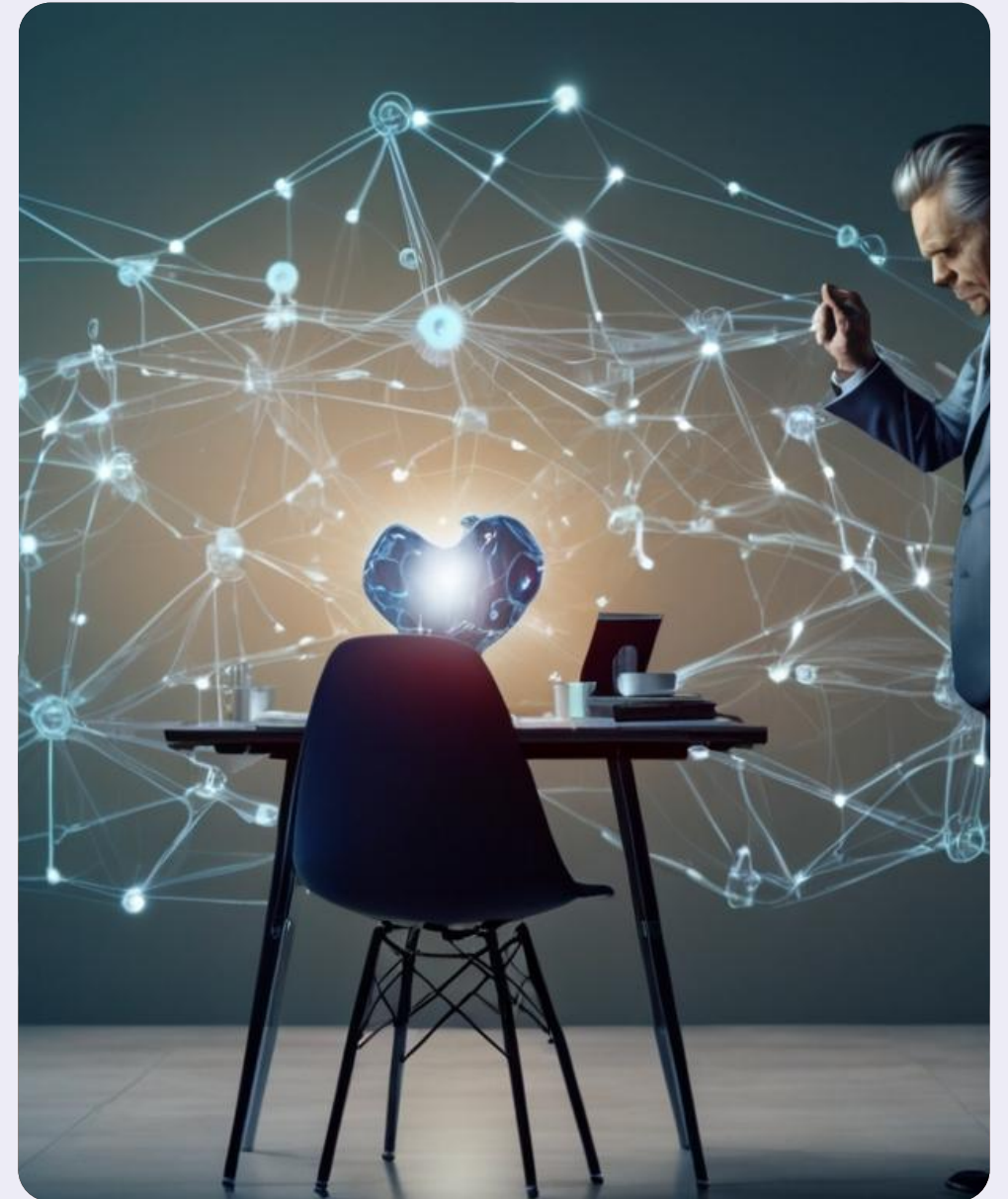




Carmelo Iaria - The Agentic Architect Playbook

Module 2

Collaboration Patterns, Risk Controls, and Governance



Governance Through Context Engineering

When Quality and Speed are on the same side

01

From **foundations** (Module 1)

Understanding the Agentic Architect role

02

To **governance at speed** (Module 2)

Operationalizing quality through context

03

Focus today: **Context Layer**

The engine of quality and compliance

We'll make governance **proactive, repeatable, and codified**



Poll Question - Current Role

What is your current role?

- Solution Architect
- Enterprise AI Leader
- Technical Program Manager
- Software Engineer Manager
- AI Consultant



Poll Question - Tools

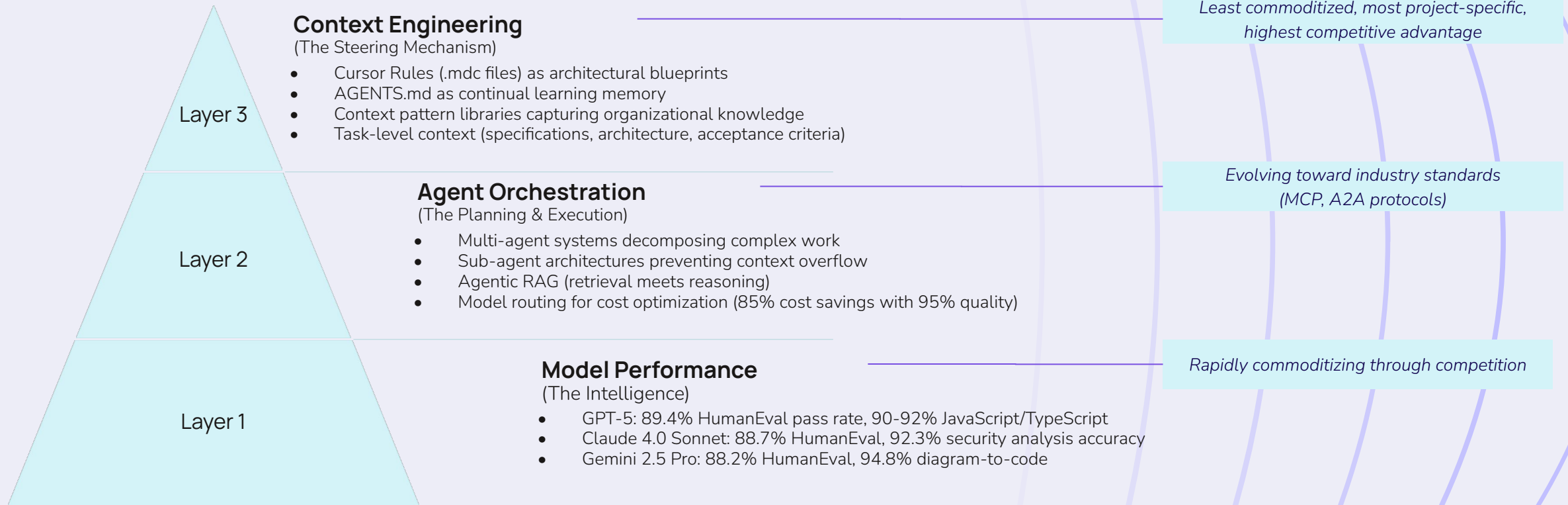
How many of you use AI Assisted Code Development Tools **daily**?

- Cursor
- Claude Code
- GitHub Copilot
- Antigravity
- Other



The Three-Layer Framework for Agentic Code Quality

Peak Performance Requires Optimization Across All Three



❏ Organizations implementing all three framework layers report **55% faster task completion** and **26% more completed tasks**.



The Three-Layer Framework for Agentic Code Quality

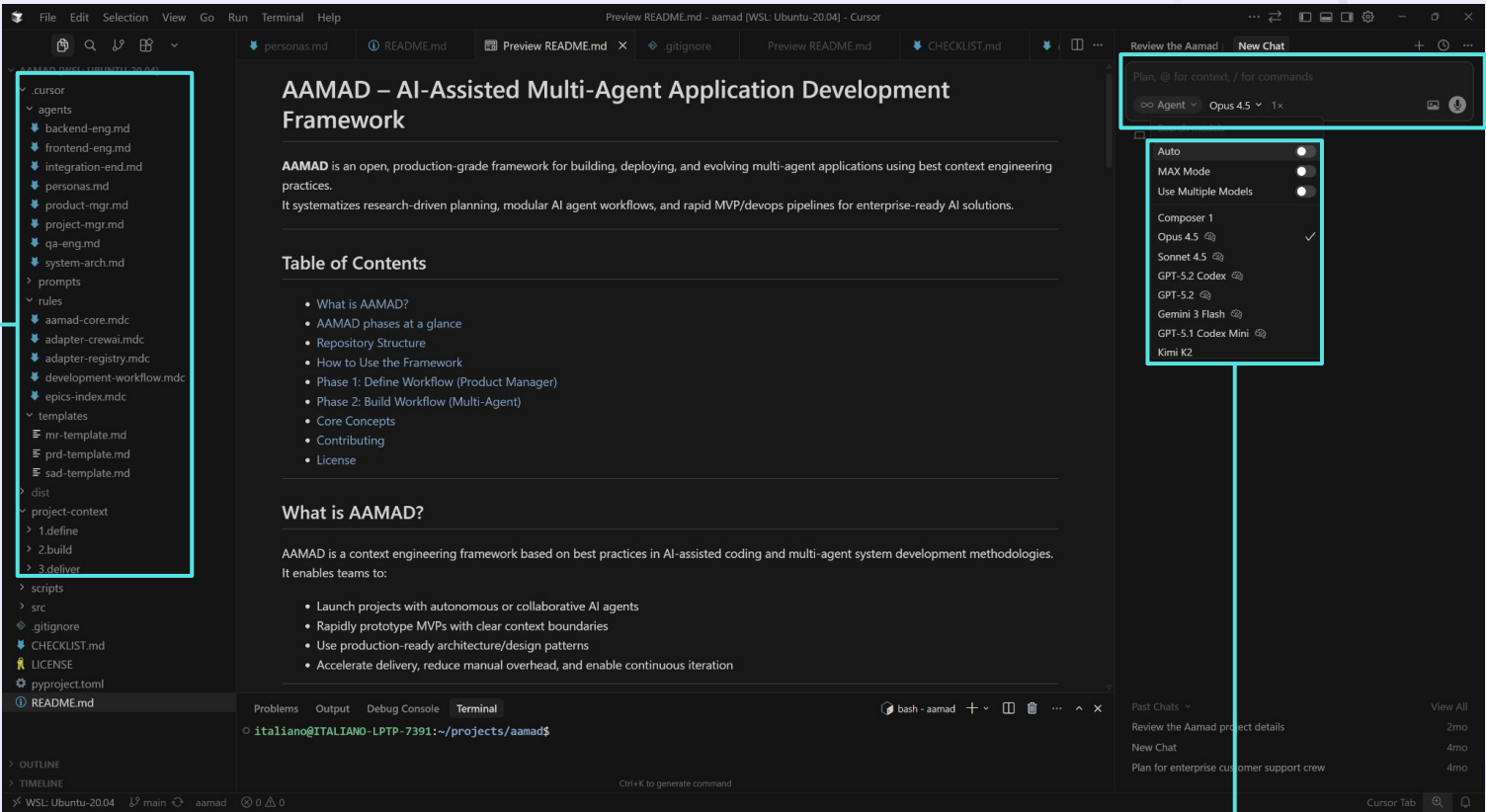
How AI Assisted Code Development has evolved and how to ensure quality while leveraging the speed



The Three-Layer Framework for Agentic Code Quality

What does it look like in the real world?

Context Engineering
(The Steering Mechanism)



Agent Orchestration
(The Planning & Execution)

Model Performance
(The Intelligence)



Why Governance Fails Without Context

Speed Without Context = Fast Slop

Agents can now:

- Edit hundreds of files autonomously
- Run swarms that coordinate complex changes

Without context engineering, they:

- Reintroduce old patterns
- Break architecture boundaries
- Violate security/compliance rules

Core Problem:

Governance is often treated as **policy documents** and **code reviews**, not as **machine-readable constraints**.



The Context Layer as Governance Engine

Context Engineering = Governance as Code

Context engineering manages:

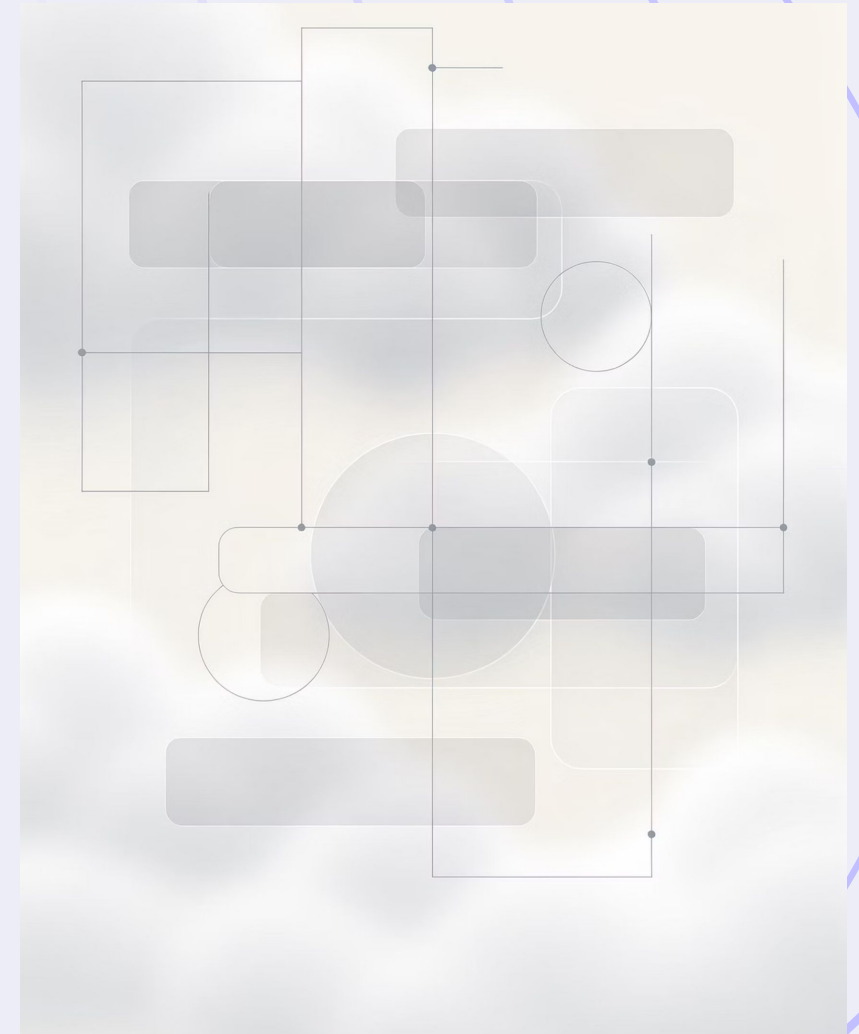
- System instructions
- Rules and conventions
- External knowledge (KBs, APIs, diagrams)
- Output structures

It is:

- Least commoditized layer
- Most project-specific
- Highest leverage for Agents & Models

Governance Angle:

Encode "how we build" and "what's allowed" into: Rules files, Templates, Agent personas, Pattern libraries



From Reactive to Proactive Governance

What Changes When Context Is First-Class



Reactive Governance (Today)

- Human reviews catching issues after the fact
- Security/compliance teams as bottlenecks
- Repeated mistakes across teams and repos



Proactive Governance (Context-Driven)

- Agents start from org patterns, not internet defaults
- Guardrails embedded in prompts, rules, and templates
- Validated patterns reused across all projects

Outcomes:

Fewer policy violations

Less manual review

Faster approvals (trust in the system)



Our Use Case Revisited – Governance Lens

Customer Support Multiagent System

CAPSTONE USE CASE

The Scenario

A mid-market bank wants to automate 70% of customer support through a multi-agent system while maintaining human oversight for complex cases.

What responses are **never** allowed?

(e.g., PII leakage, investment advice)

How do we ensure **auditability**?

Of every automated decision

How do we keep agents **aligned**?

With brand tone and regulations

How do we prevent unsafe **changes**?

From swarms making code modifications



Context Artifacts Overview – What We'll Build

CAPSTONE USE CASE

Four Pillars of Context for Governance

Cursor Rules

1

(.mdc files) Project-wide and feature-specific

Example: `customer-support-agents.mdc`

Code & API Templates

2

TypeScript handler template for support API

Logging & audit wrapper snippet

AI Agent Personas

3

(.md files) specifying agents personas

Will show examples from the AAMAD* Framework

Context Pattern Library

4

(.md files) defining context patterns for specific domain

Example: `context-patterns/customer-support.md`

* AI-Assisted Multi Agent Development



Cursor Rule Basics – Governance in the Editor

Cursor Rules as Architectural Blueprint

CAPSTONE USE CASE

1

Design Principles:

- Write what TO do, not what NOT to do
- Provide concrete examples
- Keep rules focused and composable
- Version-control rules alongside code

Rules levels:

- User rules (global)
- Project rules (.cursor/rules/*.mdc)
- Local/manual rules (@ruleName)

Precedence:

- Local/manual (@rule)
- Auto-attached by glob
- Agent-requested
- Always

.cursor/rules/customer-support-agents.mdc

```
---
name: customer-support-agents
scope: project
attach:
  - "src/support/**"
  - "infra/support/**"
---

# Project Overview
You are contributing to the Customer Support Agent Ecosystem for a regulated mid-market bank.

Core goals:
- Automate ~70% of Tier-1 and Tier-2 support interactions
- Maintain full auditability and regulatory compliance (KYC, AML, data privacy)
- Preserve brand voice: empathetic, professional, non-promissory

# Architectural Principles
- All agent-facing code must:
  - Log decisions with correlation IDs
  - Avoid direct access to raw PII; use tokenized IDs
  - Route high-risk intents (disputes, fraud, credit decisions) to human escalation

# Coding Standards
- Language: TypeScript for services, React for internal tools
- Error handling: never swallow errors; log with `supportLogger` and return typed error objects
- Observability: use `auditTrail.logDecision(...)` for all automated resolutions

# Compliance Constraints
- Never generate content that:
  - Provides investment advice
  - Confirms or denies account balances in free text
  - Always include `complianceTag` on decisions requiring audit review

# Example
When implementing a new resolution handler, follow:
- `src/support/handlers/template/ResolutionHandlerTemplate.ts`
- `src/support/policies/riskClassification.ts`
```



Code Template Embedding Governance

Resolution Handler for Support Requests

Benefits:

- Strong typing and clear contracts
- Risk-first control flow
- Privacy by design
- Auditability and observability baked in
- Separation of policy vs. business logic
- Consistency for humans and AI in Cursor

src/support/handlers/template/ResolutionHandlerTemplate.ts

```
// ResolutionHandlerTemplate.ts
// Use this template for all new automated resolution handlers

import { classifyRisk } from ".../policies/riskClassification";
import { auditTrail } from ".../observability/auditTrail";
import { supportLogger } from ".../logging/supportLogger";

export interface ResolutionContext {
  correlationId: string;
  customerId: string; // tokenized identifier, NEVER raw PII
  channel: "chat" | "email" | "phone";
  intent: string;
  metadata: Record;
}

export interface ResolutionResult {
  type: "AUTOMATED" | "ESCALATE";
  message: string;
  complianceTag?: "LOW" | "MEDIUM" | "HIGH";
}

export async function handleResolution(
  ctx: ResolutionContext
): Promise {
  const risk = classifyRisk(ctx);

  // Log input at structured, non-PII level
  supportLogger.info({
    event: "resolution:start",
    correlationId: ctx.correlationId,
    intent: ctx.intent,
    channel: ctx.channel,
    risk,
  });

  if (risk === "HIGH") {
    const result: ResolutionResult = {
      type: "ESCALATE",
      message: "This request requires review by a human specialist.",
      complianceTag: "HIGH",
    };
  }
}
```



Context Pattern Library

.cursor/support/customer-support-patterns.md

CAPSTONE USE CASE

4

```
---
name: customer-support-patterns
description: >
  Operational patterns for a banking customer support crew and LLM agents.
  Defines safe behaviors, Do/Don't rules, and escalation triggers for
  balances, fraud, and access issues.
tags:
  - customer_support
  - banking
  - safety
  - escalation
scope:
  - chat
  - email
  - phone
---
```

Context Pattern Library - Customer Support Agent Ecosystem

Pattern 1: SAFE_BALANCE_INQUIRY

Intent:

- Customer asks about account balance or "how much money" they have.

Use when:

- The customer asks about balance, funds available, or "can I afford X".
- The request is not clearly fraud-related or an account dispute.

Do:

- Use internal API `GET /balances/{customerTokenId}` via secure backend, never from the client.
- Respond with categorized information, not exact amounts, in chat channels.
- Example phrasing: "Your checking account is in good standing and above your configured alert threshold."

Don't:

- Expose raw balance numbers directly in LLM responses.
- Log full balances in application logs or analytics.
- Infer or guess balances without calling the backend.

Escalate when:

- Customer disputes a transaction or says a charge is incorrect.
- Multiple failed authentication attempts are mentioned or detected.
- Any signals suggest account takeover (e.g., "I can't recognize recent activity").

Agent notes:

- Prefer qualitative descriptions over numbers.
- If unsure whether to show an exact amount, escalate instead of guessing.
- Always respect channel: chat responses should be more abstract than secure portal views.

Pattern 2: FRAUD_SUSPICION

Intent:

- Customer suspects fraud, unauthorized use, or identity theft.

Use when:

- The customer uses words like "stolen", "identity theft", "card cloned", "unauthorized charge", "someone used my card".
- The message suggests they did not perform a transaction.

Do:

- Immediately classify risk as HIGH.
- Generate an escalation case using the `AGENT\_Escalation` output template.
- Flag the case with `fraud\_suspected=true` for the dedicated fraud team.
- Provide calm, action-oriented language (e.g., "I will open a fraud review case now and connect you to our specialist team.").

Don't:

- Attempt to definitively reassure or deny fraud (no "this is definitely not fraud").
- Suggest waiting without taking concrete action.
- Ask for sensitive PII like full SSN, full card number, or passwords.

Escalate when:

- ANY fraud keywords appear ("stolen", "identity theft", "card cloned", "unauthorized").
- The customer expresses fear about identity theft or account takeover.
- Internal risk systems or prior notes mark the account as under review.

Agent notes:

- Treat all fraud mentions as high priority and time-sensitive.
- Keep responses short, clear, and focused on next steps and escalation.
- Never promise outcomes (e.g., refunds) that require human review.

Pattern 3: PASSWORD_AND_ACCESS_ISSUES

Intent:

- Customer cannot log in, forgot password, or has trouble accessing their account.

Use when:

- The customer mentions login failures, password resets, locked accounts, or 2FA issues.

Do:

- Use approved flows and URLs from `auth\_flows.md`.
- Provide clear, step-by-step instructions that mirror the official recovery process.
- Use only links from the approved domain list (e.g., `https://bank.example.com/...`).
- Encourage customers not to share passwords or codes with anyone.

Don't:

- Ask for full SSN or complete card numbers.
- Provide login or reset links that are not on the approved domain list.
- Suggest any workaround that bypasses the official authentication flow.

Escalate when:

- The customer reports being locked out after following the official steps.
- There are multiple failed authentication attempts or suspicious access patterns.
- The access issue appears related to potential fraud or account takeover.

Agent notes:

- Always default to security over convenience.
- If unsure whether a URL is approved, omit it and escalate instead of improvising.
- Keep instructions aligned with `auth\_flows.md`, never invent new steps.



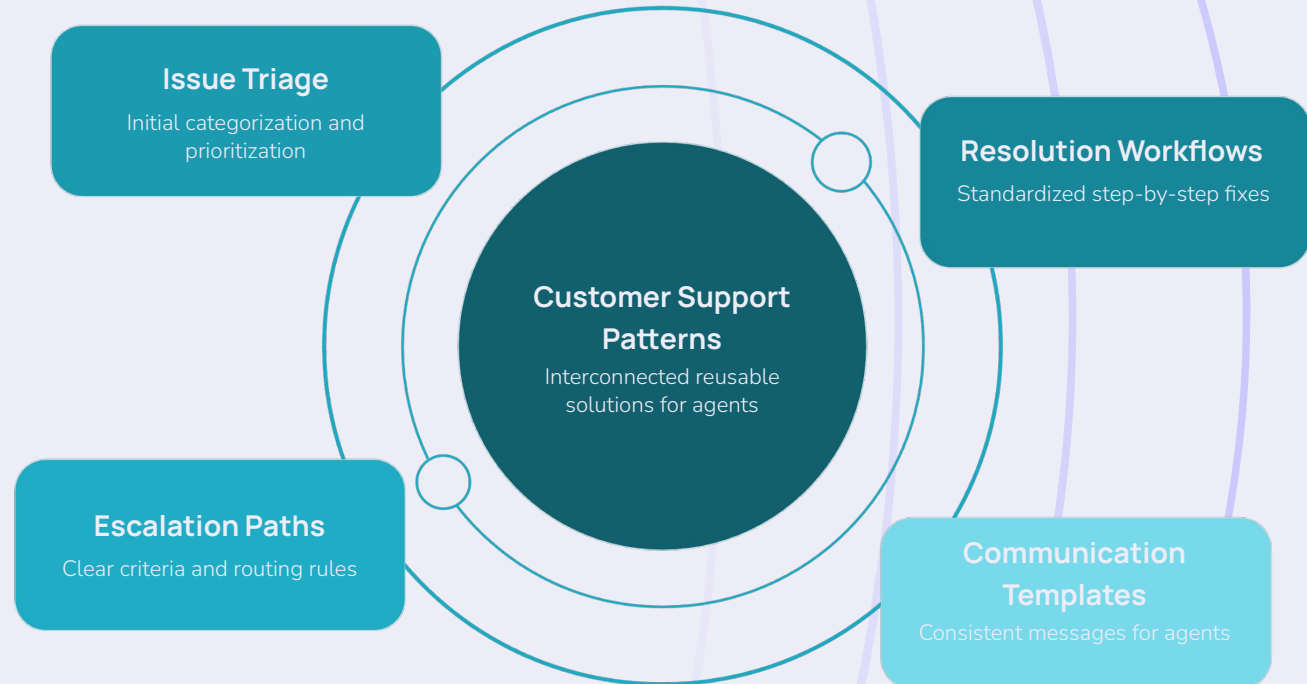
Pattern Library as a Knowledge Map

Resolution Handler for Support Requests

CAPSTONE USE CASE

4

A well-structured pattern library provides a comprehensive, interconnected guide for agents. Instead of fragmented documents, patterns are categorized and linked, enabling quick discovery and consistent application of best practices.



AI Agent Personas: The AAMAD Approach

Structured Workflow Ownership for Predictable Agentic Behavior

In the AAMAD (AI Assisted Multi Agent Development) framework, AI Agent Personas are more than just descriptions; they are **persona-driven development entities** where each workflow is owned and documented by a clear AI agent following the single responsibility principle. Defined in `.cursor/agents/` as markdown files, these personas act as codified contracts, ensuring alignment with organizational goals and safety protocols, and enabling autonomous or collaborative AI agent execution.



Single Responsibility & Clear Roles

Each persona embodies a specific role (e.g., Product Manager, System Architect) and owns specific epics/workflows with clear boundaries.



Explainability Through Artifacts

All actions, decisions, and documentation are stored as versioned markdown artifacts, ensuring traceability, reproducibility, and auditability.



Phased Execution & Parallelization

Personas operate across distinct phases (Define, Build, Deliver), making development parallelizable while retaining quality control.



Reusable & Extendable Patterns

They provide reusable and extendable patterns for agentic behavior, maintaining clear context boundaries and single responsibility across the framework.



AAMAD Framework: Three-Phase Workflow

Structured Workflow Ownership for Predictable Agentic Behavior

AAMAD organizes work into three phases: Define, Build, and Deliver, each with clear artifacts, personas, and rules to keep development auditable and reusable. The flow begins by defining context and templates, proceeds through multi-agent build execution, and finishes with operational delivery.



AAMAD Agent Personas: Roles & Responsibilities

Structured Workflow Ownership for Predictable Agentic Behavior

01

Phase 1: Define

Product Manager

- Conducts prompt-driven discovery and context setup
- Generates Market Research Document (MRD)
- Creates Product Requirements Document (PRD)
- Produces comprehensive context handoff artifacts for technical teams
- Validates completeness of market analysis, user personas, feature requirements, and success metrics

Outputs stored in `project-context/1.define/`

02

Phase 2: Build

Project Manager

- Coordinates multi-agent workflow execution
- Manages epic sequencing and dependencies
- Ensures artifact versioning and traceability

System Architect

- Generates solution architecture document (SAD)
- Defines technical stack and system design patterns
- Establishes architectural boundaries and integration points

Frontend Engineer

- Builds UI components and user interfaces
- Creates placeholder implementations
- Documents frontend architecture and patterns

Backend Engineer

- Implements backend services and APIs
- Develops data models and business logic
- Documents backend architecture and integration points

Integration Engineer

- Wires up communication between frontend and backend
- Implements chat flows and data pipelines
- Verifies end-to-end connectivity
- Documents integration patterns

QA Engineer

- Performs end-to-end testing
- Logs test results, bugs, and limitations
- Validates quality metrics and acceptance criteria
- Documents QA findings and recommendations

Outputs stored in `project-context/2.build/`

03

Phase 3: Deliver

DevOps Engineer

- Manages continuous deployment pipelines
- Configures hosting environments
- Implement access control and security measures
- Handles release management and runtime concerns
- Documents deployment procedures and operational runbooks

Outputs stored in `project-context/3.deliver/`



Developing a Multi Agent Application with AAMAD

Sample end-to-end journey



Learn AAMAD

GitHub Page

Free Self-paced mini-course to learn how to apply AAMAD to bootstrap your AI Assisted Code Development projects

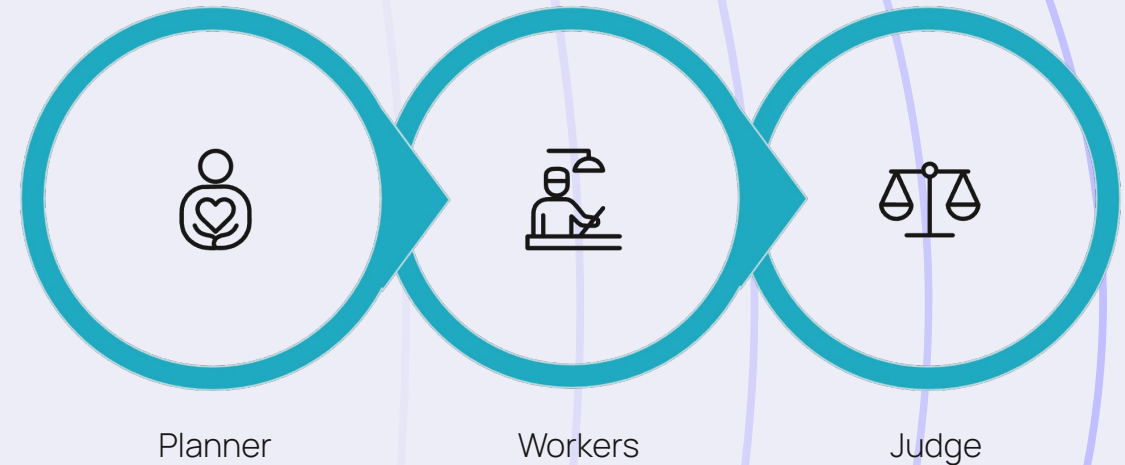


The Agent Swarm in Action

How Context Guides Autonomous Execution

From Rules & Templates to Swarm Behavior

-  **Planner agent:**
Reads `.mdc` project rule + context pattern library. Designs plan for code changes
-  **Worker agents:**
Use templates (`ResolutionHandlerTemplate.ts`) to implement code. Follow agent personas when generating logic around risk, escalation
-  **Judge/evaluator agent:**
Checks outputs against rules & patterns. Flags deviations before PR creation



Result:

Governance enforced at: Planning stage (planner uses rules), Implementation stage (workers use templates), Validation stage (judge applies patterns)



Proactive Governance Across Projects

Scaling Governance Beyond a Single Use Case

Mechanism:

Organization-wide rule and pattern repos:

- `org-rules/security.mdc`
- `org-rules/logging.mdc`
- `org-patterns/customer-support.md`
- `org-patterns/onboarding.md`
- Project-specific overrides and extensions
- Shared AGENT templates for common personas

Benefits:

- New projects inherit governance by default
- Fewer one-off, inconsistent implementations
- Faster onboarding for new teams and agents



Quality & Speed – Metrics to Track

How Context-Driven Governance Shows Up in the Numbers

Quality Metrics

- Reduction in policy violations per release
- Decrease in production incidents tied to agent behavior
- Increase in code review "pass on first review" rate

Speed Metrics

- Time from requirement to merged PR for governed areas
- Time to roll out changes across multiple products (via shared patterns)
- Percentage of new handlers built from templates vs. from scratch

Strategic Metrics

- Number of teams reusing org-wide patterns
- Reduction in manual compliance review hours
- Faster certification cycles (SOC 2, ISO, etc.)



The Agentic Architect's Role in the Context Layer

Where YOU Make the Difference

You Are Accountable For:

Designing the rule hierarchy (org vs. project vs. local)

Defining agent personas for key workflows

Curating pattern libraries from real incidents and successes

Ensuring templates reflect current best practices

You Are Not:

- Manually enforcing every rule
- Personally reviewing every PR
- Writing all application code

Your Impact:

One well-crafted `.mdc` file can influence thousands of lines of code

One good pattern can eliminate entire classes of bugs

One set of personas can anchor safety across multiple crews



Anti-Patterns – How Context Goes Wrong

Common Failures in Context Engineering

1. Monolithic Rule Files

2,000+ lines, everything in one place → agents ignore or misapply

2. Negative Instructions Only

"Never do X, don't do Y" → LLMs struggle, often do exactly that

3. Stale Context

Rules not updated as architecture changes → conflicting guidance

4. Hidden Governance

Policies in PDFs & wikis, not in `.mdc` or patterns

5. Copy-Paste Context

Teams duplicating patterns across repos → inconsistency, drift

Mitigations:

- Keep rules small, focused, composable
- Emphasize DO patterns
- Regular context reviews tied to architecture changes
- Single source of truth repos for rules and patterns



Module 2 Wrap-Up & Bridge to Module 3

From Governance to Observability

Key Takeaways:

- Context Layer is the **governance engine** of the three-layer framework
- Governance becomes **proactive** when encoded in rules, templates, personas, and patterns
- For our customer support use case, we defined: Project-level Cursor rule, Agent personas, Code templates and Context Pattern library
- Agentic Architects differentiate themselves by **owning the context layer**

Next Module (3 – Observability & Evaluation):

- How do we see what agents are doing in production?
- How do we **evaluate** quality at model, agent, and system levels?
- How do we connect observability back to **continuous context improvement**?

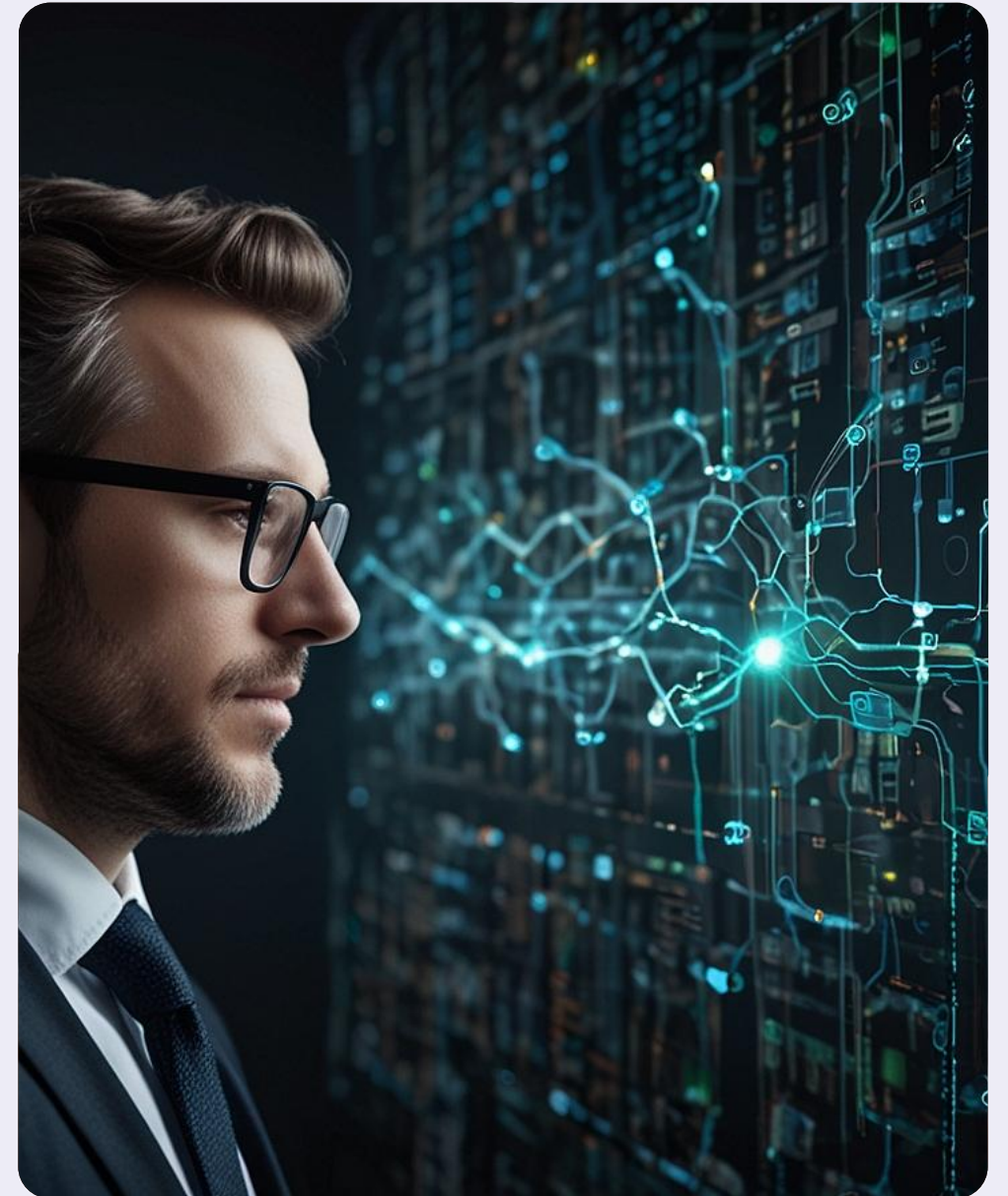




Carmelo Iaria - The Agentic Architect Playbook

Module 3

Observability, Evaluation, and ROI Frameworks



Poll Question - AI Impact

Which Layer will provide the biggest economic impact?

- Foundational Models
- Agentic Development
- Context-based Governance
- Agentic Application
- Human Curiosity



From Execution to Measurement

From governance (Module 2) → to observability & evaluation (Module 3)

Focus: Seeing what agents do, measuring impact, proving ROI

I'll guide you to build dashboards and evaluation frameworks for the customer support use case

📌 Key Insight: Module 2 showed how to design systems. Module 3 shows how to operate them. You can't improve what you don't measure. This is where trust with stakeholders gets built—through visibility and evidence.



The Observability Problem – What Agents Hide

Agents Create Opacity at Scale

Thousands of Invisible Decisions

Agents make thousands of decisions daily with no human visibility

What You Lose Without Logging

- Root cause analysis when things go wrong
- Evidence of compliance adherence
- Performance baselines for improvement
- Audit trails for regulatory requirements

The Production Lie

"It works in production" is a lie without observability

Core Question: If an agent makes a bad decision and no log captures it, did the decision really happen?

Observability is not optional—it's foundational. In Module 2, we encoded governance. Module 3 makes it verifiable.



The Three-Layer Evaluation Framework

Quality Signals at Every Level

Layer 1: Model Performance

- Accuracy on narrow tasks (classification, extraction)
- Factual correctness (hallucination rates)
- Response quality on known benchmarks
- Cost per inference

Model layer = table stakes (most vendors similar)

Layer 2: Agent Performance

- Plan quality (did the agent choose the right steps?)
- Tool selection accuracy (did it call the right API?)
- Multi-step success rates (did it complete the task end-to-end?)
- User satisfaction with outputs

Agent layer = differentiation (orchestration & planning)

Layer 3: System Performance

- End-to-end workflow completion
- Latency and resource utilization
- Cost per resolution
- Business KPIs (NPS, CSAT, revenue impact)

System layer = business value (what the company actually cares about)

📌 **Strategic Insight:** Most teams obsess over Layer 1 (model benchmarks). Smart teams measure all three simultaneously. You need all three to make coherent improvements.



Observability Architecture for Agentic Systems

The Observability Stack

01

Structured Logging

Tools: Datadog, New Relic, Grafana Loki

- Every agent decision logged with correlation ID
- Request/response payloads at key decision points
- Compliance-relevant events flagged

03

Metrics

Tools: Prometheus, Datadog

- Success rates by intent type
- Latency percentiles (p50, p95, p99)
- Cost metrics (API calls, tokens, compute)

02

Traces

Tools: OpenTelemetry, Jaeger

- Full execution path of multi-step workflows
- Timing for each sub-agent call
- Error propagation and escalation triggers

04

Evaluation Logs

Tools: Custom or via Langsmith/Evals

- Model outputs evaluated against rubrics
- Agent plans scored for soundness
- System outcomes compared to baselines

*This is standard platform observability, with AI-specific additions. The key is **correlation IDs**—connecting user request to model response to business outcome.*



Logging Structure – The Audit Trail

What Gets Logged and Why

Why This Structure

correlationId = trace entire customer journey

decision object = evaluate agent reasoning

cost = calculate ROI

complianceTag = filter for audit requirements

This structured logging is essential for making agents trustworthy by enabling analysis.

For Every Agent Decision:

```
{
  "timestamp": "2026-01-25T10:30:00Z",
  "correlationId": "cust-support-abc123",
  "agentName": "TriageAgent",
  "input": {
    "intent": "billing"
  },
  "decision": {
    "target": "AUTO"
  },
  "output": {
    "routingTarget": "ResolutionAgent"
  },
  "cost": {
    "modelName": "claude-opus"
  },
  "complianceTag": "LOW"
}
```



Traces – Following the Multi-Agent Journey

OpenTelemetry for Agentic Workflows

Key Benefits

Spot Slow Agents

Pinpoint underperforming sub-agents.

Identify Costs

Monitor expensive operations to optimize spend.

Debug Failures

Rewind traces to diagnose and fix issues.

Correlate Outcomes

Link latency directly to business results.

Traces transform "the system is slow" into actionable insights like "ResolutionAgent context retrieval is taking too long," enabling direct optimization.

Example: Customer Inquiry to Resolution

```
START: Customer message received
├─ SPAN: TriageAgent
│   ├── INPUT: message + metadata
│   ├── MODEL_CALL: claude-opus
│   └─ DURATION: 2.3s
├─ SPAN: ResolutionAgent (nested)
│   ├── CONTEXT_RETRIEVAL: Knowledge base search
│   ├── MODEL_CALL: claude-opus
│   └─ DURATION: 1.5s
└─ END: Response sent to customer
    TOTAL_DURATION: 4.8s
    TOTAL_COST: $0.0047
```



Metrics Dashboard – Real-Time Signals

Key Metrics for Customer Support Agents

On-Dashboard in Real-Time:

Metric	Target	Alert Threshold
Queries Processed/Hour	800+	< 600
Auto-Resolution Rate	70%	< 65%
Escalation Rate	25%	> 30%
Avg Resolution Time (auto)	< 3s	> 5s
Escalation Handoff Quality	> 90%	< 85%
Model Cost per Resolution	< \$0.01	> \$0.02
Triage Accuracy	> 94%	< 90%
Compliance Violations	0	> 0 (immediate alert)



Dashboard Layout

How to Organize Your Metrics

01

Top row: Business KPIs

Queries/hour, resolution rate, cost

02

Middle row: Quality metrics

Triage accuracy, handoff quality

03

Bottom row: Operational health

Latency, error rates, compliance



Executive Communication: These metrics are what you show stakeholders weekly. They translate agent behavior into business language.



Evaluation Layer 1 – Model Performance

Testing Narrow Tasks Systematically

What We Evaluate:

Triage Accuracy

Does the agent correctly classify intent?

- Test set: 200 queries
- Target: > 94% accuracy
- Regression test: new models must exceed baseline

Response Factuality

Does the agent generate factually correct responses?

- Test set: 50 known scenarios
- LLM-as-judge (0-5 scale)
- Target: > 4.2/5 average

Compliance Adherence

Does the agent generate forbidden content?

- Test set: 30 adversarial prompts
- Red-team evaluation
- Target: 100% rejection

Automation

- Run weekly on production model
- Compare to previous week
- Regression → immediate escalation

Foundation Principle: Layer 1 is baseline hygiene. If the model fails here, nothing else matters.



Evaluation Layer 2 – Agent Orchestration Quality

Testing Multi-Step Plans and Tool Use

What We Evaluate:



Plan Soundness

Does the agent break down the task correctly?

- 40 complex inquiries
- Expert evaluation (rubric, 0-5 scale)



Tool Selection Accuracy

Did the agent call the right API/tool?

- 60 diverse scenarios
- Automated check: expected vs. actual API
- Target: >96% accuracy



Error Recovery

Does the agent escalate gracefully upon API failure?

- Inject failures in 20% of requests
- Expect human escalation or graceful retry

Simulation Harness

Synthetic personas run pre-recorded scenarios automatically.

500+ nightly scenarios

Regression → investigation

Rollback if needed

❏ Orchestration Insight: Layer 2 reveals coordination failures, saving downstream review time.



Evaluation Layer 3 – System & Business Outcomes

Measuring What Actually Matters

First Contact Resolution (FCR)

% of inquiries resolved without escalation

Target: 70%

Tracked weekly by intent type

Customer Satisfaction (CSAT)

Post-resolution survey: "Was your issue resolved?" (0-5)

Separated: auto-resolved (target 4.2+) vs. escalated (target 3.8+)

Track trends by agent type, intent, channel

Cost Per Resolution

(Model cost + infrastructure + oversight) /
resolved

Target: < \$0.05 (including human escalation costs)

Tracked vs. human baseline (\$2-3 per resolution)

Compliance & Risk

Policy violations per 100K resolutions

Target: < 1

Any violation triggers investigation + root cause analysis

Agent Accuracy Over Time

Track per-agent and per-version accuracy

Identify: which agents are drifting?

Trigger retraining or context updates



System & Business Outcomes

Example Monthly Report

24K

Queries Processed

January 2026

70%

Auto-Resolved

16,800 queries

30%

Escalated

7,200 queries

\$0.043

Avg Cost/Resolution

vs. target \$0.05 ✓

Quality Metrics

CSAT (auto): 4.3 (vs. target 4.2) ✓

CSAT (escalated): 3.9 (vs. target 3.8) ✓

Compliance violations: 0 ✓

Business Impact

\$187,000

Cost saved vs. human-only

Executive Insight: Layer 3 is what you present to CFO and board. This is proof the system is creating value.



Human-in-the-Loop Evaluation

Where Humans Review Agent Decisions

Sampling Strategy

Random Sampling

2% of all escalations (140 per month)

Purpose: detect drift, edge cases, policy violations

Risk-Stratified Sampling

100% of HIGH-risk decisions

Purpose: ensure no critical breaches

Trend-Based Sampling

All disputes, fraud flags

Purpose: catch systemic issues early

Evaluation Rubric (for each reviewed case)

Decision Quality (0-3):

- 0: Incorrect routing
- 1: Correct outcome but poor reasoning
- 2: Correct outcome with acceptable reasoning
- 3: Correct outcome with excellent reasoning

Compliance (0-2):

- 0: Policy violation detected
- 1: Edge case, needs guideline clarification
- 2: Fully compliant

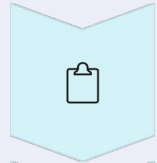
Agent Improvement Opportunity: Yes/No

Yes → suggest context update, template fix, or persona refinement



Feedback Loop

From Evaluation to Improvement



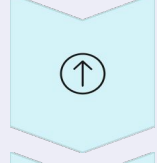
Human feedback logged with correlation ID



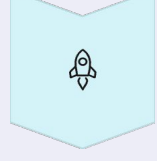
Weekly review: aggregate feedback → identify patterns



Monthly: prioritize context/template improvements



Update .mdc files, agent personas, templates



Deploy updated context → agents improve automatically

Continuous Improvement: This closes the loop: observation → evaluation → improvement. Humans aren't bottlenecks; they're the improvement engine.



Continuous Evaluation – Detecting Drift

Production Monitoring for Agent Degradation

Detection Mechanisms

Statistical Process Control

- Track weekly: auto-resolution, CSAT, escalation accuracy
- Alert if metrics drop > 5% WoW
- Example: escalation rate jumps 25% to 31%

Anomaly Detection

- Isolation Forest on decision logs
- Flag agents making abnormal decisions
- Example: agent escalating low-risk cases

LLM-as-Judge Regression

- Weekly re-evaluation of past resolutions
- Investigate if quality scores drop
- Rollback model if needed

Customer Complaint Analysis

- Monitor "bad advice" escalations
- Correlate to agents/contexts
- Trigger retraining or context updates

Action Triggers

Minor drift (< 2%)

Log & monitor

Moderate drift (2-5%)

Investigate + review

Major drift (> 5%)

Pause, rollback, incident response

Early Warning System: Catch drift before it becomes a crisis.



ROI Calculation – Quantifying Business Value

How to Prove This Investment Pays Off

Baseline (Human-Only Support)

- 24,000 queries/month
- Cost per resolution: \$2.50 (agent labor + overhead)
- Monthly cost: \$60,000
- CSAT: 3.6
- FCR: 65%

With Agentic System (70% automation target)

- 24,000 queries/month
- 16,800 auto-resolved: \$0.05 per resolution = \$840
- 7,200 escalated to human: \$1.50 per resolution = \$10,800
- Monthly cost: \$11,640
- CSAT: 4.1 (improved from 3.6)
- FCR: 70%

Monthly ROI

\$48K

Monthly Savings

\$60,000 - \$11,640

81%

Cost Reduction

vs. human-only baseline

4

Months to Breakeven

On \$200K system development



Year 2+ ROI

Long-Term Value Realization

- No development costs
- Operations + maintenance: \$200,000/year
- Annual savings: $\$720,000 - \$200,000 = \$520,000$
- ROI: 260%



Long-Term Value: First year ROI is often breakeven or slightly negative. Real value emerges in year 2 and beyond. Don't just show cost; show customer satisfaction and competitive advantage too.



Customer Satisfaction & Net Promoter Score

Beyond Cost – Measuring Loyalty

Baseline (Human Support)

- NPS: 32 (neutral)
- CSAT (human-resolved): 3.6

With Agentic System

- CSAT (auto-resolved): 4.3
- Blended CSAT: 4.1
- NPS: 42 (+10 points)

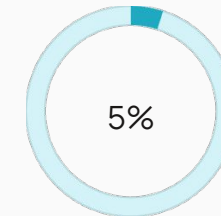
Converting NPS to Revenue

- 10-point NPS increase correlates to 5-10% revenue lift (industry dependent)

Strategic Value

- Faster Support & Retention
- Better First-Contact Resolution & Advocacy
- 24/7 Availability & Competitive Edge

For a bank with \$50M customer book:



Conservative Revenue Lift

$5\% \times \$50M = \$2.5M$ additional revenue



5-Year Impact

$\$2.5M \times 5 = \$12.5M$ incremental revenue



Operational Metrics Dashboard

The Single Pane of Glass

Dashboard Sections (in Datadog, Grafana, or Looker):

Section 1: Volume & Efficiency

- Queries processed (daily, weekly, monthly trend)
- Auto-resolution rate (target: 70%, alert if < 65%)
- Avg resolution time by path (auto: 3s, escalated: 45s)

Section 2: Quality

- CSAT (auto vs. escalated, by intent)
- Escalation accuracy (% of escalations actually needing humans)
- Triage accuracy (% correctly classified)

Section 3: Agent Health

- Per-agent success rates (detect underperformers)
- Knowledge base freshness (when was it last updated?)
- Context version and deployment status

Section 4: Trend Analysis

- Weekly auto-resolution rate (detect drift)
- CSAT trend (up or down?)
- Cost trend (improving or degrading?)

This daily dashboard provides a weekly executive brief for stakeholders.



Improvement in Action

Real-World Example

Example Improvement



Metrics on Improvement Velocity

- Bugs detected (via eval) → fix deployment: target < 48 hours
- Context drift detected → correction deployed: target < 1 week
- Customer satisfaction feedback → action: target 2 weeks

"Continuous Improvement in Practice: This is what continuous improvement actually looks like. Without measurement, improvement is just hope."



Module 3: Summary & Next Steps

From Theory to Actionable Insights

Module 3 provided a comprehensive framework for ensuring the reliability, performance, and business value of agentic systems. We explored four critical pillars:



Observability

Implementing detailed logging, tracing, and real-time metrics dashboards to gain transparency into multi-agent operations.



Evaluation

A three-layered framework for assessing agent performance, orchestration quality, and overall system effectiveness, including human-in-the-loop processes.



ROI Calculation

Quantifying the tangible business value through cost savings, improved customer satisfaction (NPS), and reduced compliance risks.



Continuous Improvement

Establishing robust feedback loops and weekly cycles to detect drift, root cause failures, and continuously refine agent behavior and context.

Looking Ahead: Operationalizing for Impact

As we move to Module 4, we will shift our focus from measurement and evaluation to the practical aspects of **deploying, managing, and scaling agentic systems responsibly** in real-world environments. This includes understanding the operational best practices, governance structures, and the human element in successful AI integration.

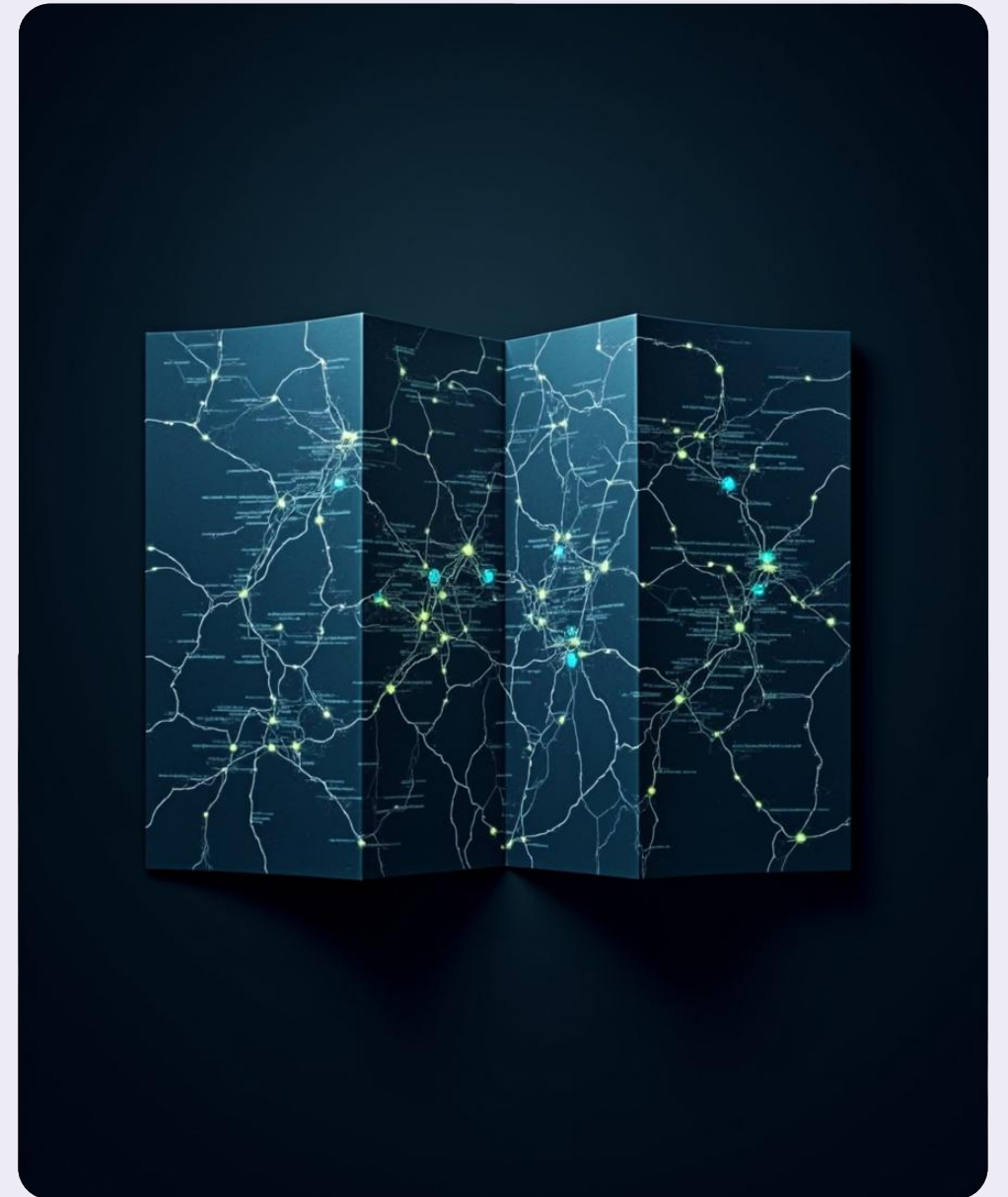




Carmelo Iaria - The Agentic Architect Playbook

Module 4

Rollout Strategy, Change Management, and Capstone Review



Poll Question - AI Adoption

What percentage of **large enterprise** will have transformed **most** of their internal processes with Agentic AI Automation **by 2030**?

- 5%
- 25%
- 45%
- 65%
- 85%



Poll Question - AI Adoption

What percentage of **small & medium business** will have transformed **most** of their internal processes with Agentic AI Automation **by 2030**?

- 5%
- 25%
- 45%
- 65%
- 85%



From Blueprint to Production

From **measurement** (Module 3) → to **execution and rollout** (Module 4)

01

Real-world deployment

Moving from design to live production systems

02

Change management

Getting real humans to use what you've built

03

Capstone implementation

Your complete rollout blueprint for customer support

We'll build a one-page rollout blueprint and execution timeline for the customer support use case. You've learned the role, designed governance, built observability—now comes the hardest part: getting real humans to use what you've built. This module is about de-risking the rollout and maximizing adoption.



The Rollout Paradox – Why Good Systems Fail

Smart Design ≠ Successful Adoption

Key Failures:

- System works perfectly in pre-prod
- Stakeholders trained and on board
- Launch day arrives... and teams bypass the agents
- "It's easier to just handle it myself"
- "I don't trust the agent yet"
- "The agent doesn't understand my specific case"

Root Causes:

- Insufficient change management
- Unclear roles and incentives post-launch
- No "quick wins" early to build confidence
- Rollout too fast → insufficient learning time
- Rollout too slow → momentum lost

This is where many AI projects die—not in design, but in adoption. Change management is 50% of your job as Agentic Architect.



The One-Page Rollout Blueprint

Your Master Plan at a Glance

Phasing

3-4 waves over 12 weeks

Success Criteria

Metrics that define "this is working"

Key Stakeholders

Who owns what

Risk Mitigation

What could go wrong and how we'll handle it

Communication Plan

What we tell people and when

Format:

- Fits on a single page (landscape, 11pt font)
- Updated weekly
- Posted in Slack, shared in standup
- Becomes the single source of truth for the entire org

This replaces 47-slide project decks that nobody reads. Clear, concise, visible—the mark of a confident plan.



Phase 1 – Internal Validation (Weeks 1-3)

Proof of Concept with Your Team

Objectives:

- Validate that agents work as designed
- Catch integration bugs before external users
- Build internal confidence in the system

Activities:

- Customer support team (10-15 people) uses agents in parallel with human support
- Agents handle real tickets; humans review 100% of outputs
- Collect feedback on agent quality, UX, integration issues

Success Criteria:

- ✓ Zero critical bugs (data corruption, PII leakage, escalation failures)
- ✓ Triage accuracy > 90% (may not hit 94% yet, that's OK)
- ✓ CSAT on reviewed outputs > 3.8/5
- ✓ Support team gives thumbs-up to rollout (survey: > 70% confidence)

Key Metrics to Track:

- Bug reports per 100 tickets (target: < 2)
- Time to human review per ticket (target: < 5 min)
- Team sentiment on Slack and surveys

Stakeholders:

Owner: Support Operations Manager

Champion: Senior support agent (respected by team)

Technical Lead: You (Agentic Architect)

Phase 1 is about finding what you missed, not proving success. Expect to find bugs and fix them fast. Build momentum by shipping improvements weekly.



Phase 2 – Soft Launch (Weeks 4-6)

Limited Production with Safety Nets

Objectives

- Deploy agents to production on a subset of traffic
- Measure real-world performance vs. lab estimates
- Refine based on actual customer interactions

Activities

- Route 25% of incoming tickets to agents
- 75% still go to humans (control group for comparison)
- Monitor dashboards hourly; be ready to dial back if needed
- Daily standup to review metrics and incidents

Success Criteria

- ✓ Auto-resolution rate: 55-65% (ramping up to 70% target)
- ✓ Escalation accuracy: > 85%
- ✓ CSAT (auto-resolved): > 4.0/5
- ✓ Zero regulatory/compliance breaches
- ✓ System latency: p95 < 5s

Rollback Plan

- If CSAT drops below 3.8 → investigate, don't roll back immediately
- If compliance violations appear → immediate pause + root cause
- If latency p95 > 10s → investigate infrastructure, possible rollback

Stakeholders

Owner: Support Operations Manager
Decision Maker: VP of Customer Success
Technical Lead: You

This phase is your reality check. Expect performance to be 85-90% of lab estimates; that's normal. Focus on learning, not perfection.



Phase 2 – Continuous Improvement Cycle

How to Improve During Soft Launch

Example Improvements

Week 1: "Agents don't understand card fraud signals" → update fraud pattern in

AGENT_Triage.md +
context-patterns/fraud.md

Week 2: "Escalation handoff missing customer history" → improve **AGENT_Escalation** output template

Week 3: "Billing inquiries frequently routed to wrong specialist" → refine intent classification in **riskClassification.ts** and **.mdc** rules

Deployment Strategy

- All updates go through Git → code review → staged deployment
- Use feature flags to roll back instantly if needed
- New context available to agents immediately (no model retraining)

Communication

- Friday recap email: "Here's what we fixed this week"
- Show the team their feedback drove improvements
- Build confidence: "We're listening and responding fast"

This is where the context layer pays dividends. You can fix issues in hours/days, not weeks. Transparency builds trust.



Phase 3 – Ramping (Weeks 7-10)

Growing Confidence and Volume

Objectives:

- Gradually increase agent traffic share
- Monitor metrics continue to improve or stabilize
- Train full support team on new workflows

Activities:

- Week 7: 40% of tickets to agents
- Week 8: 55% of tickets to agents
- Week 9: 65% of tickets to agents
- Week 10: 70% of tickets to agents (full target)
- Daily standups shift to every-other-day as stability increases

Success Criteria:

- ✓ Auto-resolution rate tracking toward 70%
- ✓ Escalation accuracy: > 90%
- ✓ CSAT (auto-resolved): > 4.2/5
- ✓ No new incident categories emerging
- ✓ Support team comfort level: survey > 85%
- ✓ Cost per resolution: trending toward \$0.05

Training Activities:

- Week 7: "How agents work + how to escalate effectively" (30 min workshop)
- Week 8: "Reading agent logs for quality review" (hands-on lab)
- Week 9: "Giving feedback to improve agent behavior" (1:1 coaching)

Stakeholders:

- Owner: Support Operations Manager
- Training Lead: Whoever manages support onboarding
- Executive Sponsor: VP of Customer Success (weekly updates)

This phase is about building operational muscle. You're shifting from "let's be careful" to "this is how we work now."
But you're not over the finish line yet.



Phase 4 – Stabilization & Optimization (week 11-12)

From Launch to Steady State



Objectives

Reach full production traffic mix, establish steady-state operations and runbooks, transition from launch team to permanent ops team



Activities

100% of tickets routed based on agent triage/escalation logic, on-call support for agent issues, monthly metrics reviews instead of weekly, knowledge base updates on cadence



Success Criteria

Metrics stable at target levels (70% auto-resolution, 4.2 CSAT, \$0.05 cost), support team independently improving agents, compliance sign-off, executive approval

Operations Handoff:

- Document runbooks (what to do when agent fails)
- Establish escalation path (who to call when things break)
- Build alerting rules (what triggers wake-up calls)
- Define SLOs (what level of reliability we expect)

Stakeholders:

Owner: Transitions to Support Operations Manager (fully)

Architect Role: Shifts to "advisor" from "operator"

Ops Team: Now owns on-call and day-to-day monitoring

Success here means you can step back. This isn't abandonment—you're still available for major improvements. But the system no longer needs constant care.



Risk Mitigation – The "Oh No" Plan

What Could Go Wrong and How We'll Handle It

Risk	Impact	Probability	Mitigation
Agent generates PII in response	Catastrophic	Low	100% human review in Phase 1, sampling thereafter; immediate escalation if detected
Escalation routing fails; customers stuck in queue	High	Medium	Feature flag to revert to 100% human routing; 30-min alert threshold
CSAT drops below 3.5	High	Low	Investigate; if context issue, rollback context change; if model issue, revert to previous model
Support team refuses to use agents (adoption failure)	High	Medium	Have a "skeptic" on team; involve them in improvements; celebrate wins publicly
Agent latency degrades (p95 > 30s)	Medium	Low	Scale infrastructure; profile slow agents; optimize context retrieval
New policy announced mid-rollout; agents violate it	High	Low	Immediate context update + human review layer until validated



Incident Response Plan

Preparing for clear communication

Incident Response:

P0 (data breach, regulatory violation): Pause agents immediately, investigate, brief executive team

P1 (major CSAT drop, adoption crisis): Emergency standup within 2 hours, communication plan within 4 hours

P2 (minor issues, performance dips): Root cause analysis within 24 hours, fix within 48 hours

Communication in Crisis:

- Honesty: "Here's what happened. Here's what we're doing."
- Speed: "We'll update you every 4 hours."
- Competence: Show you understand the problem and have a plan.

Hope is not a plan. De-risking means naming risks and handling them head-on.
The fact that you have a plan makes people trust you more, even when things go wrong.



The Communication Plan

What to Tell People and When



Over-communication beats under-communication. Different audiences need different stories. Transparency on challenges builds more trust



The Capstone Artifact

One Page Rollout Blueprint

CUSTOMER SUPPORT AGENT SYSTEM – ROLLOUT BLUEPRINT

Timeline: January 25 – April 22, 2026 | Owner: Sarah Chen, Ops Manager

PHASE TIMELINE:

— PHASE 1: INTERNAL VALIDATION (Weeks 1-3: Jan 25 – Feb 14)| Objective: Proof of concept with 15 internal users
| Success Criteria: Zero critical bugs | Triage accuracy >90% | Team confidence
| Key Metrics: Bug rate < 2/100 tickets | Human review time < 5 min
| Owner: Support Operations Manager | Champion: Senior Support Agent

— PHASE 2: SOFT LAUNCH (Weeks 4-6: Feb 15 – Mar 7)| Objective: 25% production traffic with safety nets
| Success Criteria: AutoRes 55-65% | CSAT >4.0 | Zero breaches | p95 latency <5s |
| Key Metrics: Daily dashboard review | Weekly improvements deployed
| Rollback Plan: If CSAT < 3.8 or breach detected, pause agents

— PHASE 3: RAMPING (Weeks 7-10: Mar 8 – Apr 4)| Objective: Gradual traffic increase to 70% with team training
| 40% (Wk7) – 55% (Wk8) – 65% (Wk9) – 70% (Wk10)
| Success Criteria: AutoRes 70% | CSAT >4.2 | Team survey >85% confidence
| Training: "How agents work" – "Log reading" – "Feedback coaching"

— PHASE 4: STABILIZATION (Weeks 11-12: Apr 5 – Apr 22)| Objective: 100% production + ops handoff
| Success Criteria: Metrics stable | Compliance sign-off | Arch → Advisor role
| Handoff: Runbooks | On-call rotations | Alert rules | SLOs

RISK MITIGATION:

P0 Risks (Catastrophic)
[PII leakage in responses]
→ 100% human review Phase 1
• Regulatory breach
→ Feature flag for human mode

P1 Risks (High Impact)
[CSAT drop below 3.5]
→ Investigate, context roll-back if needed
• Escalation queue failures
→ 30-min alert, dial down

P2 Risks (Medium)
[Latency degradation]
→ Scale infrastructure
• Agent adoption failure
→ Involve skeptics in improvements

WEEKLY METRICS DASHBOARD:

Auto-Resolution: 70% (target) | CSAT (auto): 4.2/5 | Cost/Resolution: \$0.05
Escalation Accuracy: 90%+ | Compliance Violations: 0 | System Uptime: 99.5%+

COMMUNICATION CADENCE:

Weekly: Friday 5-min Slack update (metrics, wins, blockers)
Monthly: All-Hands 10-min update (business impact stories)
Monthly: Steering Committee (metrics, roadmap, risks)
On-Incident: Transparency updates every 4 hours

DEPENDENCIES & APPROVALS:

- ✓ Executive Sponsor: VP Customer Success
- ✓ Technical Lead: Agentic Architect (You)
- ✓ Compliance Officer: Sign-off on governance controls
- ✓ CFO: Approval of \$200K investment with ROI targets
- ✓ Support Team: Consensus on approach

DEFINITION OF SUCCESS:

- ✓ 70% of tickets auto-resolved by Week 10
- ✓ CSAT on auto-resolved tickets ≥ 4.2/5
- ✓ Cost per resolution ≤ \$0.05
- ✓ Zero compliance breaches throughout rollout
- ✓ 80%+ team adoption survey by end of Phase 3
- ✓ NPS improvement: +10 points year 1
- ✓ Projected savings: \$520K annually (Year 2+)

KEY CONTACTS:

Operational Issues: Sarah Chen (sarah@bank.com) | On-call rotation
Technical Issues: You (architect@bank.com) | Advisor on-demand
Escalations: VP Customer Success | CC on all P1 incidents
Compliance: Chief Compliance Officer | Governance oversight

NEXT STEPS (Before Phase 1 Kickoff on Jan 25):

- Executive team alignment meeting (Jan 22)
- Support team training & questions (Jan 23)
- Final system validation & monitoring setup (Jan 24)
- Phase 1 kickoff celebration (Jan 25)

Last Updated: January 25, 2026 | Next Review: February 1, 2026



Change Management – The Human Side

Adoption Is Harder Than Technology

1. Involve Early (Weeks 1-3)

Get 2-3 vocal support agents on the design team from day 1. Their feedback shapes the product. They become champions who say "I helped build this"

2. Show the Jobs Agent Will Do First (Week 3)

"Agent will handle 70% of routine inquiries: billing, card issues, password resets." "You'll handle 30% of complex cases: disputes, fraud, complaints." "Your job shifts from routine to judgment-based"

3. De-Risk with Soft Launch (Weeks 4-6)

Agents review every auto-response (builds confidence). Escalations come back to you fast (no one stuck in a queue). You're not competing; you're partner with the agent

4. Celebrate Wins Publicly (Every phase)

"Agents caught 3 fraudulent transactions this week that slipped human review." "Customer NPS improved because of faster resolution." "Team freed up 20 hours this week for complex cases"

5. Offer Career Pathways (Ongoing)

"Agent quality reviewer" (evaluate agent outputs, give feedback). "Agent behavior coach" (help improve agent decision-making). "AI operations specialist" (manage agent monitoring and escalations). These are promotions, not demotions



Change Management – The Human Side

Adoption Is Harder Than Technology

Handling Resistance:

The Skeptic: Involve them early. Their concerns are often valid.

The Avoider: Pair them with an agent "buddy." Start with low-stakes cases.

The Saboteur (rare): Have a private conversation. "Here's why this is happening. Here's your role."



Core Truth: Support agents are skilled professionals doing complex work. Agent rollout feels like "we're replacing you." They need to believe: "This makes my job easier, not harder."

Change management often determines success or failure more than technology. Treat your team as partners, not resistance to overcome.



Metrics-Based Course Correction

How to Know When to Pivot

If Auto-Resolution Rate Lags (stuck at 60% instead of 70%)

Investigate: Are agents routing too conservatively?

Action: Review escalation decisions, update

`riskClassification.ts`, lower HIGH-risk thresholds for low-impact intents

Owner: You (Agentic Architect)

If CSAT Drops Below 4.0

Investigate: Which intent types are problematic? Is it auto-responses or escalations?

Action: Human evaluation of failing cases, update response templates or personas

Owner: Support Operations Manager + You

If Escalation Accuracy Falls Below 85%

Investigate: Are we escalating too much (false positives) or not enough (false negatives)?

Action: Retrain triage agent or adjust confidence thresholds

Owner: You + Domain expert (compliance officer, support lead)

If Cost Per Resolution Exceeds \$0.07

Investigate: Are context retrievals slow? Are we using expensive model calls?

Action: Optimize knowledge base queries, consider cheaper model for triage vs. resolution

Owner: You (technical lead)

If Team Adoption Survey Stays Below 70%

Investigate: What specific concerns? Gather qualitative feedback in 1:1s.

Action: Targeted improvements (faster escalations, better agent transparency, career paths)

Owner: Support Operations Manager + You

Metrics-driven doesn't mean rigid. If data shows you need to change course, change course. Ego is the enemy of good projects.



The Team You Need to Succeed

You Can't Do This Alone

Core Team (Phases 1-4, Weeks 1-12):

You (Agentic Architect / Technical Lead): Strategy, design, governance, context

Operations Manager: Day-to-day execution, team communication, change management

Support Engineer (champion): Testing, feedback, internal champion

Compliance Officer (part-time): Risk review, policy alignment

Product Manager (part-time): Roadmap, success criteria, stakeholder management

Extended Team (Months 2-12):

Data Analyst: Metrics, dashboards, trend analysis

ML/AI Engineer (if model fine-tuning needed): Model optimization, evaluation

Junior Architect (hire after Months 3): Patterns documentation, next domain handoff

Support Trainer: Change management, team skill-building

Hiring for Your Success:

Trait 1: Systems thinkers (they understand the whole picture, not just their part)

Trait 2: Learn-it-yourself people (AI is moving fast; they need to stay current)

Trait 3: Comfort with ambiguity (first-of-a-kind projects are messy)

Trait 4: Customer empathy (everything we do is to help customers)

Building Your Network:

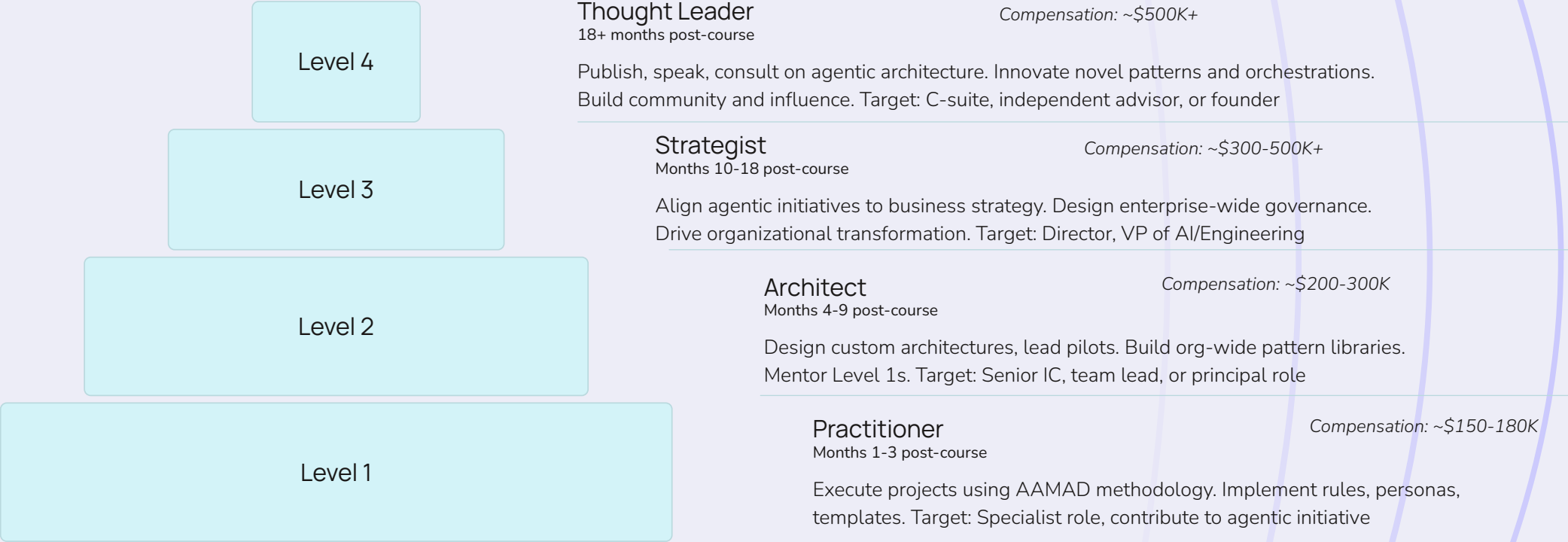
- Find other Agentic Architects in your industry; share learnings
- Join communities: LangChain, Claude Developers, Anthropic forums
- Attend conferences: AI Engineer Summit, LLM Systems Workshop
- Contribute back: Write about your journey, help others

You're not a solo hero; you're a connector. Your job is to elevate your team, not to be the smartest person in the room.



The Agentic Architect's Career Beyond This Course

Skill Progression & Opportunities



This is a high-demand, high-growth career. Your 2-year career progression could normally take 8-10 years. First-mover advantage is real.



The Journey You've Completed

From Student to Agentic Architect

Module 1: Foundations

- ✓ Understood the role (coordination, not code)
- ✓ Learned the competencies (systems thinking, context engineering, orchestration, HITL, evaluation, governance, business acumen)
- ✓ Saw market timing (first-mover advantage, talent shortage)
- ✓ Identified your use case (customer support bank ecosystem)

Module 3: Observability & Evaluation

- ✓ Designed observability architecture (logging, traces, metrics, evals)
- ✓ Implemented three-layer evaluation (model, agent, system)
- ✓ Built ROI model (81% cost reduction, 10-point NPS improvement, 260% Year 2+ ROI)
- ✓ Created feedback loops (measurement → improvement → action)

Module 2: Governance & Context Engineering

- ✓ Built `.mdc` rules (machine-readable governance)
- ✓ Designed agent personas (specialization via context)
- ✓ Created code templates (governance as default pattern)
- ✓ Built pattern libraries (reusable, org-wide best practices)

Module 4: Delivery & Rollout

- ✓ Built a one-page rollout blueprint (phasing, risks, communication)
- ✓ Designed change management (adoption, careers, celebration)
- ✓ Planned continuous improvement (metrics-driven iteration)
- ✓ Mapped your future (optimization, expansion, thought leadership)

You started as a student and now you have everything you need to be the architect. The hardest work is ahead (execution), but you're prepared.



Bringing It Home – Your 90-Day Action Plan

From Classroom to Organization

1

Weeks 1-2 (Immediately)

- Share Module 1 with your manager: "Here's why this matters"
- Identify your use case: Where in your org could this work?
- Audit existing systems: Do you have governance gaps? Observability gaps?
- Find allies: Who in your org "gets it"?

2

Weeks 3-4

- Draft a one-page rollout blueprint for your use case
- Share it with stakeholders: "Here's the plan. What are your concerns?"
- Build your governance artifacts (.mdc rules, personas, templates)
- Design your observability stack

3

Weeks 5-8

- Begin Phase 1 with a small internal team
- Implement weekly feedback loops (measure, improve, iterate)
- Start documenting your patterns (build your moat)
- Celebrate wins publicly (build momentum)

4

Weeks 9-12

- Launch Phase 2 (soft launch to production)
- Run your weekly standups, metric reviews
- Scale to Phase 3, then Phase 4
- Plan your next domain (expansion)

90-Day Success Looks Like:

- ✓ Internally, agents are handling 25-40% of your use case
- ✓ Your team believes in the system (70%+ in surveys)
- ✓ You have concrete ROI data (cost savings, CSAT improvement, customer feedback)
- ✓ Leadership is asking: "When can we do this for our other problems?"
- ✓ You're recognized as the expert on agentic architecture in your org

The next 90 days are critical. This window determines whether you build momentum or lose it. Start small, be transparent, celebrate wins.



The Bigger Picture – Why This Matters

Beyond Your Organization

The Inflection Point We're At:

- AI capabilities are now good enough for autonomous operation
- Most organizations are in "pilot purgatory" (many pilots, few in production)
- The gap isn't intelligence—it's orchestration, governance, observability
- Organizations that master agentic architecture will dominate their industries by 2027-2028

Your Role in This Transformation:

- You're not just building systems; you're helping your organization evolve
- You're creating models where humans and agents collaborate optimally
- You're proving that AI augments (not replaces) human judgment
- You're building competitive advantages that are defensible and sustainable

The Competitive Moat:

- Your context patterns and playbooks become your IP
- Your team's expertise in agentic architecture becomes irreplaceable
- Your ability to ship features 3x faster than competitors becomes your advantage
- Your first-mover success stories become your recruiting narrative

The Career Arc:

- 18 months ago: "AI? That's for researchers"
- Today: "I'm learning to architect AI systems"
- 18 months from now: "I'm the person who scaled agentic architecture at our company"
- 3 years from now: "I'm recognized as a leader in this field"

This is a pivotal moment in software history. Your choices in the next 6-12 months will define your career for the next decade.
The opportunity is real; the window is open.



Your Capstone Commitment

From Foundations to Flight

What You've Learned:

- The Agentic Architect role: coordination and governance, not just code
- How to design systems that are safe, observable, and compliant
- How to prove business value through ROI measurement
- How to execute rollouts that actually get adopted

What You'll Build:

- A complete agentic system from scratch
- Governance artifacts that scale across your organization
- Observability that gives you visibility into every decision
- An organization that ships agentic products at speed with confidence

Your Commitment (Post-Course):

- I commit to building agentic systems using the AAMAD framework (Alignment, Architecture, Measurement, Decision)
- I commit to owning governance, not delegating it
- I commit to transparency (good and bad) with my stakeholders
- I commit to continuous learning (models, techniques, patterns evolve fast)
- I commit to mentoring others and building community

The Payoff:

For your career: Rare, high-demand skills commanding premium compensation

For your org: 50-80% cost reduction, improved CSAT, competitive advantage

For your industry: Contributing to responsible, governed AI adoption

For yourself: Being part of the inflection point where AI-native systems become the default



Closing thoughts

"The best time to plant a tree was 20 years ago. The second-best time is now."

Your organization will adopt agentic systems. The question is: will you lead that transformation or react to it? You now have the knowledge and the frameworks. The only thing left is execution.

You've got this. 🚀



The Agentic Architect Playbook

Managing the Enterprise Transformation in the
Intelligence Revolution age



Carmelo Iaria

Founder, Synaptic AI Consulting



APPENDIX: Resources for Post-Course Learning



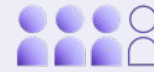
Documentation to Read

- Anthropic's "Building effective agents" guide
- LangChain Agent documentation and examples
- OpenAI Assistants API best practices



People to Follow

- Harrison Chase (LangChain founder)
- Patrick Choisnard (context engineering pioneer)
- Sam Altman, Demis Hassabis (industry leaders)



Communities to Join

- LangChain Discord (agent discussions)
- Anthropic's Claude Developers community
- /r/LanguageModels and /r/OpenAI (Reddit)



Your Next Deep Dives

- Advanced orchestration patterns (hierarchical planning, network-based coordination)
- Fine-tuning models for your domain
- Advanced evaluation (using stronger models as judges)
- Building custom tools and APIs for your agents

